
Server code running locally

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import { loginUser } from "../api/auth";

export default function LoginForm() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const navigate = useNavigate();

  const decodeJwt = (token) => {
    try {
      const base64Url = token.split(".")[1];
      const base64 = base64Url.replace(/-/g, "+").replace(/_/g, "/");
      const jsonPayload = decodeURIComponent(
        atob(base64)
          .split("")
          .map((c) => "%" + ("00" + c.charCodeAt(0).toString(16)).slice(-2))
          .join("")
      );
      return JSON.parse(jsonPayload);
    } catch {
      return {};
    }
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    setError("");

    try {
      const result = await loginUser({ email, password });

      if (!result.token) {
        setError("Invalid credentials");
        return;
      }

      // store token
      localStorage.setItem("token", result.token);
    }
  };
}
```

```

const decoded = decodeJwt(result.token);
const isOrgPresent = decoded?.is_org_present ?? false;
const isAdmin = decoded?.is_admin ?? false;

if (isOrgPresent) navigate("/dashboard");
else if (isAdmin) navigate("/orgRegister");
else navigate("/dashboard");
} catch (err) {
  setError(err.message || "Login failed");
} finally {
  setLoading(false);
}
};

return (
  <div style={{ maxWidth: "400px", margin: "50px auto" }}>
    <h2>Login</h2>
    {error && <p style={{ color: "red" }}>{error}</p>}
    <form onSubmit={handleSubmit}>
      <div>
        <label>Email</label><br />
        <input
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          required
          style={{ width: "100%", padding: "8px", marginBottom: "10px" }}
        />
      </div>
      <div>
        <label>Password</label><br />
        <input
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          required
          style={{ width: "100%", padding: "8px", marginBottom: "10px" }}
        />
      </div>
      <button type="submit" disabled={loading}>
        {loading ? "Logging in..." : "Login"}
      </button>
    </form>
  </div>
);
}

```

```
import api from "../axios";
```

```
export const loginUser = async (data) => {
  try {
    const response = await api.post("/auth/login_user", data);
    return response.data;
  } catch (err) {
    throw err.response?.data || err;
  }
};
```

```
import axios from "axios";

const api = axios.create({
  baseURL: "/api", // use Vite proxy
  headers: { "Content-Type": "application/json" },
  withCredentials: true, // include cookies
});

// Automatically attach token from localStorage
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export default api;
```

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'
// https://vite.dev/config/
export default defineConfig({
  plugins: [react(),
    tailwindcss()
  ],
  server: {
    proxy: {
      "/api": {
        target: "http://66.116.197.167:5001/api/v1/hrm",
        changeOrigin: true,
        secure: false,
        rewrite: (path) => path.replace(/^Vapi/, ""),
      },
    },
  },
})
// vite.config.js
```

```
import { useState } from 'react'
import reactLogo from './assets/react.svg'
import viteLogo from '/vite.svg'
import LoginPage from './pages/LoginPage';
import Dashboard from './components/Dashboard';
import Rightpanel from './components/Rightpanel';
import './App.css'

function App() {
  return (
    <>
      <LoginPage />
      { /* <Rightpanel /> */ }
    </>
  )
}

export default App
```

```
// main.jsx or index.jsx
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { BrowserRouter } from "react-router-dom";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

export const loginAPI = (payload) => { return api.post("/auth/login", payload); }; // LOGIN ONLY
 export const loginUser = async (data) => { const response = await api.post("/auth/login_user", data); return response.data; };
 whats the difference between both?

Both functions are doing an API call to log a user in, but they differ in **how they return and expose the response**.

1. loginAPI

```
export const loginAPI = (payload) => {
  return api.post("/auth/login", payload);
};
```

Key points:

- Returns the **raw Axios Promise**
- Does **not await** the request
- Returns the full Axios response object:

```
{  
  data,  
  status,  
  headers,  
  config,  
  request  
}
```

- Caller must handle `.then()` or `await` and extract `.data` themselves

Usage example:

```
loginAPI(payload).then(res => {  
  console.log(res.data);  
});
```

or

```
const res = await loginAPI(payload);  
console.log(res.data);
```

2. loginUser

```
export const loginUser = async (data) => {  
  const response = await api.post("/auth/login_user", data);  
  return response.data;  
};
```

Key points:

- Uses `async/await`
- Returns **only response.data**
- Hides Axios response wrapper from the caller
- Simpler to consume

Usage example:

```
const data = await loginUser(payload);  
console.log(data);
```

Main difference

Feature	loginAPI	loginUser
API style	Promise return	async/await
Return value	Full Axios response	Only response.data
Caller responsibility	Must extract .data	Already extracted
Flexibility	More control	Simpler usage

When to use which?

- Use **loginAPI** if:
 - You need headers, status codes, or full response metadata
 - You want flexible handling in multiple places
 - Use **loginUser** if:
 - You only care about response data (most common case)
 - You want cleaner, simpler service functions
-

If you want, I can show you a **best-practice structure for API services (Axios wrapper pattern used in production apps)**.

```
import api from "/src/api/axios.js"; export const registerAdmin = async (data) => { try { const response = await api.post("/auth/register_admin", data); return response.data; } catch (error) { console.log(error); throw error; } }; export const registerOrg = async (data) => { try { const response = await api.post("/auth/register_org", data); return response.data; } catch (error) { console.log(error); throw error; } }; export const loginUser = async (data) => { const response = await api.post("/auth/login_user", data); return response.data; }; };
```

kay i have this in live cause there are three different user and i need this in axios too cause i am logiing in as admin

Yes — if you have **multiple user roles** (admin, org, user) and different login endpoints, you should keep separate API functions in your auth service file for clarity and maintainability.

You can structure it like this:

```
import api from "./axios";

// ADMIN LOGIN
export const loginAdmin = async (data) => {
  const response = await api.post("/auth/login_admin", data);
  return response.data;
};
```

```
// ORG LOGIN
export const loginOrg = async (data) => {
  const response = await api.post("/auth/login_org", data);
  return response.data;
};

// USER LOGIN
export const loginUser = async (data) => {
  const response = await api.post("/auth/login_user", data);
  return response.data;
};

And similarly for register:

export const registerAdmin = async (data) => {
  const response = await api.post("/auth/register_admin", data);
  return response.data;
};

export const registerOrg = async (data) => {
  const response = await api.post("/auth/register_org", data);
  return response.data;
};
```

You can also reduce repetition by creating a reusable helper:

```
import api from "../axios";

const postRequest = async (url, data) => {
  try {
    const response = await api.post(url, data);
    return response.data;
  } catch (error) {
    console.error(error);
    throw error;
  }
};

export const loginAdmin = (data) =>
  postRequest("/auth/login_admin", data);

export const loginOrg = (data) =>
  postRequest("/auth/login_org", data);

export const loginUser = (data) =>
  postRequest("/auth/login_user", data);
```

```
export const registerAdmin = (data) =>
  postRequest("/auth/register_admin", data);
```

```
export const registerOrg = (data) =>
  postRequest("/auth/register_org", data);
```

This is cleaner and scales better when your app grows.

```
import { useState } from "react"; import { loginUser } from "../api/auth"; export default
function LoginForm() { const [email, setEmail] = useState(""); const [password, setPassword]
= useState(""); const [loading, setLoading] = useState(false); const [showPassword,
setShowPassword] = useState(false); const handleLogin = async (e) => { e.preventDefault(); if
(!email || !password) { alert("Please fill all fields"); return; } try { setLoading(true); const
response = await loginUser({ email, password, }); console.log("LOGIN RESPONSE:",
response); // Adjust according to backend response if (response?.token) {
localStorage.setItem("token", response.token); } alert("Login successful!"); } catch (error) {
console.log(error); alert( error.response?.data?.message || "Login failed" ); } finally {
setLoading(false); } }; return ( <form onSubmit={handleLogin}> <h1 className="text-3xl
font-bold mb-2">Welcome!</h1> <p className="text-gray-500 mb-6"> Let's discover the HR
solutions in a Snap. </p> { /* Email */ <div className="mb-4"> <label className="block
text-sm mb-1"> Email </label> <input type="email" placeholder="Enter your email"
value={email} onChange={e => setEmail(e.target.value)} className="w-full border
rounded-lg p-3 focus:outline-none focus:ring-2 focus:ring-blue-400" /> </div> { /* Password
*/ <div className="mb-4"> <label className="block text-sm mb-1"> Password </label>
<div className="relative"> <input type={showPassword ? "text" : "password"}
placeholder="Enter your Password" value={password} onChange={e =>
setPassword(e.target.value)} className="w-full border rounded-lg p-3 pr-10 focus:outline-
none focus:ring-2 focus:ring-blue-400" /> <span onClick={() =>
setShowPassword(!showPassword)} className="absolute right-3 top-3 cursor-pointer" >
</span> </div> </div> { /* Remember + Forgot */ <div className="flex items-center justify-
between mb-6 text-sm"> <label className="flex items-center gap-2"> <input
type="checkbox" /> Remember me </label> <a href="#" className="text-blue-500"> Forgot
Password? </a> </div> { /* Button */ <div className="flex justify-end"> <button
type="submit" disabled={loading} className="w-full bg-gray-400 text-white py-3 rounded-lg
max-w-[90px]" > {loading ? "... " : "Log In →"} </button> </div> </form> ); } chnage this too
```

```
import { useState } from "react";
import {
  loginAdmin,
  loginOrg,
  loginUser,
} from "../api/auth";
```

```
export default function LoginForm() {
  const [role, setRole] = useState("user");

  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
```



```
const [loading, setLoading] = useState(false);
const [showPassword, setShowPassword] = useState(false);

const handleLogin = async (e) => {
  e.preventDefault();

  if (!email || !password) {
    alert("Please fill all fields");
    return;
  }

  try {
    setLoading(true);

    let response;

    // ROLE BASED LOGIN
    if (role === "admin") {
      response = await loginAdmin({
        email,
        password,
      });
    } else if (role === "org") {
      response = await loginOrg({
        email,
        password,
      });
    } else {
      response = await loginUser({
        email,
        password,
      });
    }

    console.log("LOGIN RESPONSE:", response);

    // SAVE TOKEN
    if (response?.token) {
      localStorage.setItem("token", response.token);
    }

    // SAVE ROLE
    localStorage.setItem("role", role);

    alert("Login successful!");
```

```

    } catch (error) {
      console.log(error);

      alert(
        error.response?.data?.message ||
        "Login failed"
      );
    } finally {
      setLoading(false);
    }
  };

  return (
    <form onSubmit={handleLogin}>
      <h1 className="text-3xl font-bold mb-2">
        Welcome!
      </h1>

      <p className="text-gray-500 mb-6">
        Let's discover the HR solutions in a Snap.
      </p>

      { /* ROLE SELECT */ }
      <div className="mb-4">
        <label className="block text-sm mb-1">
          Login As
        </label>

        <select
          value={role}
          onChange={(e) => setRole(e.target.value)}
          className="w-full border rounded-lg p-3 focus:outline-none focus:ring-2 focus:ring-
blue-400"
        >
          <option value="user">User</option>
          <option value="org">Organization</option>
          <option value="admin">Admin</option>
        </select>
      </div>

      { /* EMAIL */ }
      <div className="mb-4">
        <label className="block text-sm mb-1">
          Email
        </label>

        <input

```

```

        type="email"
        placeholder="Enter your email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        className="w-full border rounded-lg p-3 focus:outline-none focus:ring-2 focus:ring-blue-400"
      />
    </div>

```

```

    { /* PASSWORD */ }
    <div className="mb-4">
      <label className="block text-sm mb-1">
        Password
      </label>

      <div className="relative">
        <input
          type={showPassword ? "text" : "password"}
          placeholder="Enter your password"
          value={password}
          onChange={(e) =>
            setPassword(e.target.value)
          }
          className="w-full border rounded-lg p-3 pr-10 focus:outline-none focus:ring-2 focus:ring-blue-400"
        />

```

```

        <span
          onClick={() =>
            setShowPassword(!showPassword)
          }
          className="absolute right-3 top-3 cursor-pointer"
        >
          
        </span>
      </div>
    </div>

```

```

    { /* REMEMBER + FORGOT */ }
    <div className="flex items-center justify-between mb-6 text-sm">
      <label className="flex items-center gap-2">
        <input type="checkbox" />
        Remember me
      </label>

      <a href="#" className="text-blue-500">
        Forgot Password?
      </a>
    </div>

```

```

    </a>
  </div>

  { /* SUBMIT BUTTON */}
  <div className="flex justify-end">
    <button
      type="submit"
      disabled={loading}
      className="w-full bg-gray-400 text-white py-3 rounded-lg max-w-[120px]"
    >
      {loading ? "..." : "Log In →"}
    </button>
  </div>
</form>
);
}

```

no the server would understand donot give any select box

```

import { useState } from "react";
import { loginAdmin } from "../api/auth";

export default function LoginForm() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const [loading, setLoading] = useState(false);
  const [showPassword, setShowPassword] = useState(false);

  const handleLogin = async (e) => {
    e.preventDefault();

    if (!email || !password) {
      alert("Please fill all fields");
      return;
    }

    try {
      setLoading(true);

      // ADMIN LOGIN
      const response = await loginAdmin({
        email,
        password,
      });

      console.log("LOGIN RESPONSE:", response);
    } catch (error) {
      // Handle login error
    }
  };
}

```

```

// SAVE TOKEN
if (response?.token) {
  localStorage.setItem("token", response.token);
}

alert("Login successful!");

} catch (error) {
  console.log(error);

  alert(
    error.response?.data?.message ||
    "Login failed"
  );
} finally {
  setLoading(false);
}
};

return (
  <form onSubmit={handleLogin}>
    <h1 className="text-3xl font-bold mb-2">
      Welcome!
    </h1>

    <p className="text-gray-500 mb-6">
      Let's discover the HR solutions in a Snap.
    </p>

    { /* EMAIL */ }
    <div className="mb-4">
      <label className="block text-sm mb-1">
        Email
      </label>

      <input
        type="email"
        placeholder="Enter your email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        className="w-full border rounded-lg p-3 focus:outline-none focus:ring-2 focus:ring-
blue-400"
      />
    </div>

    { /* PASSWORD */ }
    <div className="mb-4">

```

```

<label className="block text-sm mb-1">
  Password
</label>

<div className="relative">
  <input
    type={showPassword ? "text" : "password"}
    placeholder="Enter your Password"
    value={password}
    onChange={(e) => setPassword(e.target.value)}
    className="w-full border rounded-lg p-3 pr-10 focus:outline-none focus:ring-2
focus:ring-blue-400"
  />

  <span
    onClick={() => setShowPassword(!showPassword)}
    className="absolute right-3 top-3 cursor-pointer"
  >
    
  </span>
</div>
</div>

{/* REMEMBER + FORGOT */}
<div className="flex items-center justify-between mb-6 text-sm">
  <label className="flex items-center gap-2">
    <input type="checkbox" />
    Remember me
  </label>

  <a href="#" className="text-blue-500">
    Forgot Password?
  </a>
</div>

{/* BUTTON */}
<div className="flex justify-end">
  <button
    type="submit"
    disabled={loading}
    className="w-full bg-gray-400 text-white py-3 rounded-lg max-w-[90px]"
  >
    {loading ? "..." : "Log In →"}
  </button>
</div>
</form>

```

```
);  
}
```

```
import { Box, Button, Checkbox, Flex, FormControl, FormLabel, Heading, Input, InputGroup, InputRightElement, Link, Stack, Text, IconButton, Image, FormErrorMessage, Progress, useToast, } from "@chakra-ui/react"; import { Navigate, Link as RouterLink } from "react-router-dom"; import { useState, useMemo } from "react"; import { ViewIcon } from "@chakra-ui/icons"; import hide from "../assets/hide.png"; import { useForm } from "react-hook-form"; import { useDispatch, useSelector } from "react-redux"; import { loginUserThunk } from "../slices/authSlice"; import { useNavigate } from "react-router-dom"; import { LuEye } from "react-icons/lu"; import { LuEyeOff } from "react-icons/lu"; import Cookies from "js-cookie"; const EMAIL_REGEX = /^[a-zA-Z0-9_.+]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$/; export default function LoginForm({ onSubmitSuccess } = {}) { const [showPassword, setShowPassword] = useState(false); const toast = useToast(); const dispatch = useDispatch(); const navigate = useNavigate(); const { loading, error } = useSelector((state) => state.auth); const token = Cookies.get("auth_token"); const inputStyle = { fontFamily: "Poppins", fontSize: "15px", _placeholder: { color: "gray.400", fontFamily: "Poppins", fontSize: "14px", }, h: "55px", borderWidth: "1.5px", borderColor: "#D0D5DD", borderRadius: "8px", }; // if (token) { // return <Navigate to="/dashboard" replace />; // } // react-hook-form config const { register, handleSubmit, watch, reset, formState: { errors, isSubmitting, isValid }, } = useForm({ mode: "onChange", criteriaMode: "all", defaultValues: { email: "", password: "", remember: false, }, }); // watch input values const emailValue = watch("email", ""); const passwordValue = watch("password", ""); // helper checks for "field is valid" used for progress calculation const isEmailValid = useMemo( () => EMAIL_REGEX.test(emailValue), [emailValue] ); const isPasswordValidForProgress = useMemo( () => passwordValue && passwordValue.length >= 8, [passwordValue] ); // compute progress percent (2 inputs: email + password) const progressPct = useMemo(() => { const total = 2; const completed = (isEmailValid ? 1 : 0) + (isPasswordValidForProgress ? 1 : 0); return Math.round((completed / total) * 100); }, [isEmailValid, isPasswordValidForProgress]); // API integration and show data in console const onSubmit = async (data) => { try { const loginData = { email: data.email, password: data.password }; const result = await dispatch(loginUserThunk(loginData)); if (result.meta.requestStatus === "fulfilled") { toast({ title: "Logged in", description: "You have successfully logged in.", status: "success", duration: 2500, isClosable: true, }); reset(); const token = result.payload?.token || result.payload?.data?.token || result.payload; // decode JWT payload (safe base64url decode) const decodeJwtPayload = (t) => { try { if (!t || typeof t !== "string") return null; const base64Url = t.split(".")[1]; if (!base64Url) return null; const base64 = base64Url.replace(/-/g, "+").replace(/_/g, "/"); const padded = base64 + "=".repeat((4 - (base64.length % 4)) % 4); const jsonPayload = decodeURIComponent( atob(padded).split("").map( (c) => "%" + ("00" + c.charCodeAt(0).toString(16)).slice(-2) ).join("") ); return JSON.parse(jsonPayload); } catch (e) { return null; } }; // store token if present if (token && typeof token === "string") { try { localStorage.setItem("token", token); } catch (e) { console.error("Failed to store token in localStorage:", e); } } const decoded = decodeJwtPayload(token); const isOrgPresent = decoded?.is_org_present ?? decoded?.isOrgPresent ?? false; const isAdmin = decoded?.is_admin ?? decoded?.isAdmin ?? false; if (isOrgPresent) { navigate("/dashboard", { replace: true }); } else { // Only admins should be allowed to create an organization profile. if (isAdmin) { import("../pages/Dashboard"); import("../layouts/AppLayout");
```

```

import(".././pages/OrganizationRegister"); navigate("/orgRegister", { replace: true }); } else {
// Non-admin employees should be redirected to dashboard and informed
navigate("/dashboard", { replace: true }); } } else { navigate("/orgRegister"); } if
(onSubmitSuccess) onSubmitSuccess(result.payload); // Optionally, redirect or store token
here } else { toast({ title: "Login failed", description: result.payload?.message || "Invalid
credentials.", status: "error", duration: 3500, isClosable: true, }); } } catch (err) { toast({ title:
"Login failed", description: "Something went wrong. Please try again.", status: "error",
duration: 3500, isClosable: true, }); } } return ( <Flex flexDirection="column" justify="center"
align="flex-start" w="100%" fontFamily={"Poppins"} > { /* Previously there was a progress
bar here but we are currently not rendering it.. */ <Box bg="white" mt="25px" px={{ base:
6, md: 75 }} w="100%" borderRadius="8px" boxSizing="border-box" > { /* Use a real <form>
so Enter submits */ <form onSubmit={handleSubmit(onSubmit)}> <Stack spacing={4}>
<Box> <Heading // fontSize={"20px", "22px", "24px"}> <Text> <Text> <Text> <Text>
fontFamily={"Poppins", sans-serif} fontWeight={700} color="#000000" lineHeight="40px" >
Welcome! </Heading> <Text color="#757575" fontSize="sm" fontFamily={"Poppins", sans-
serif"} > Let's discover the HR solutions in a Snap. </Text> </Box> { /* Email */ <FormControl
id="email" isValid={!errors.email} mt={0}> <FormLabel fontFamily={"Poppins", sans-serif"
fontWeight="500" fontSize="14px" color="#000000" > Email </FormLabel> <Input
type="email" placeholder="Enter your email" borderRadius={4} fontSize="14px"
{...register("email", { required: "Email is required", pattern: { value: EMAIL_REGEX, message:
"Enter a valid email address", }, })} aria-invalid={errors.email ? "true" : "false"} {...inputStyle}
/> <FormErrorMessage fontSize="12px" fontFamily={"poppins"} fontWeight={"500"}>
{errors.email?.message} </FormErrorMessage> </FormControl> { /* Password */
<FormControl id="password" isValid={!errors.password} mt={3}> <FormLabel
fontFamily={"Poppins", sans-serif} fontWeight="500" fontSize="14px" color="#000000" >
Password </FormLabel> <InputGroup> <Input type={showPassword ? "text" : "password"}
placeholder="Enter your Password" borderRadius={4} fontSize="14px"
{...register("password", { required: "Password is required", })} aria-invalid={errors.password ?
"true" : "false"} {...inputStyle} /> <InputRightElement h="full"> <IconButton type="button" //
prevent this icon from submitting the form aria-label={ showPassword ? "Hide password" :
"Show password" } icon={showPassword ? <LuEyeOff /> : <LuEye />} onClick={() =>
setShowPassword((s) => !s)} variant="ghost" size="sm" _hover={{ bg: "transparent" }}
_active={{ bg: "transparent" }} /> </InputRightElement> </InputGroup> { /* Password hint /
errors */ {errors.password?.types ? ( <Box mt={2}>
{Object.values(errors.password.types).map((msg, idx) => ( <Text key={idx} color="red.500"
fontSize="12px" fontFamily={"poppins"} fontWeight={"500"}> {msg} </Text> ))} </Box> ) : (
<FormErrorMessage fontSize={"12px"} fontFamily={"poppins"}
fontWeight={"500"}>{errors.password?.message}</FormErrorMessage> )} </FormControl>
{ /* Remember + Forgot */ <Flex justify="space-between" align="center"> <Checkbox
colorScheme="blue" {...register("remember")} sx={{ ".chakra-checkbox__control": { w:
"20px", h: "20px", }, }} > <Text fontSize="13px" fontFamily={"Poppins", sans-serif"
fontWeight="500" color="#274167" > Remember me </Text> </Checkbox> <Link
as={RouterLink} to="/forgot-password" color="#274167" fontSize="13px"
fontFamily={"Poppins", sans-serif} fontWeight="500" textDecoration="underline" > Forgot
Password? </Link> </Flex> { /* Button */ <Box display="flex" justifyContent="flex-end">
<Button type="submit" bg="#274167" color="white" fontFamily={"Poppins", sans-serif"
fontWeight="500" fontSize="14px" _active={{ bg: "#1f3143" }} _hover={{ bg: "#1f3143" }}

```



```
rightIcon={<span>→</span>} isLoading={loading} isDisabled={!isValid || loading} aria-disabled={!isValid || loading} mt={6} mb={-4} > Log In </Button> </Box> </Stack> </form> </Box> </Flex> ); } this is the login form i have ins erver
```

Your server-side login form is already correctly designed for a backend that determines the role automatically from the credentials/token.

You do **not** need a role dropdown or separate UI.

The only thing you need is:

- call the correct thunk/API
- backend returns token + role info
- frontend redirects based on decoded token or response

Your current logic already does that here:

```
const isOrgPresent =  
  decoded?.is_org_present ?? decoded?.isOrgPresent ?? false;
```

```
const isAdmin =  
  decoded?.is_admin ?? decoded?.isAdmin ?? false;
```

and:

```
if (isOrgPresent) {  
  navigate("/dashboard", { replace: true });  
} else {  
  if (isAdmin) {  
    navigate("/orgRegister", { replace: true });  
  } else {  
    navigate("/dashboard", { replace: true });  
  }  
}
```

So your frontend is fine.

What you mainly need is your API/auth layer to match this pattern.

Example:

api/auth.js

```
import api from "./axios";  
  
export const loginUser = async (data) => {  
  try {  
    const response = await api.post("/auth/login_user", data);  
    return response.data;  
  } catch (error) {  
    console.error(error);  
  }  
}
```

```

    throw error;
  }
};

export const registerAdmin = async (data) => {
  try {
    const response = await api.post("/auth/register_admin", data);
    return response.data;
  } catch (error) {
    console.error(error);
    throw error;
  }
};

export const registerOrg = async (data) => {
  try {
    const response = await api.post("/auth/register_org", data);
    return response.data;
  } catch (error) {
    console.error(error);
    throw error;
  }
};

```

Then your thunk:

authSlice.js

```

export const loginUserThunk = createAsyncThunk(
  "auth/loginUser",
  async (data, thunkAPI) => {
    try {
      return await loginUser(data);
    } catch (error) {
      return thunkAPI.rejectWithValue(
        error.response?.data || {
          message: "Login failed",
        }
      );
    }
  }
);

```

Your current form component can stay almost exactly as it is.

```

import api from "../axios"; // ADMIN LOGIN
export const loginAdmin = async (data) => {
  const response = await api.post("/auth/login_admin", data);
  return response.data;
}; // ORG LOGIN
export const loginOrg = async (data) => {
  const response = await

```

```
api.post("/auth/login_org", data); return response.data; }; // USER LOGIN export const
loginUser = async (data) => { const response = await api.post("/auth/login_user", data);
return response.data; }; currently this is in my auth.js whwere should i change to meet my
serevrs criteria cause login failed and i donot have authslice where shpould i put the file
```

Your auth.js is fine.

The issue is probably that your frontend is still calling loginUser() while your server expects /auth/login_admin.

Since you said:

"I am logging in as admin"

you should change the import + function call in your login form.

Change THIS

```
import { loginUser } from "../api/auth";
```

to:

```
import { loginAdmin } from "../api/auth";
```

Then change THIS

```
const response = await loginUser({
  email,
  password,
});
```

to:

```
const response = await loginAdmin({
  email,
  password,
});
```

You do NOT need:

- authSlice
- Redux
- thunk
- select box
- extra files

if your project is simpler.

Your folder structure should simply be:

```
src/
├── api/
│   ├── axios.js
│   └── auth.js
├── components/
│   └── LoginForm.jsx
```

Your auth.js is already correct:

```
import api from "../axios";

export const loginAdmin = async (data) => {
  const response = await api.post("/auth/login_admin", data);
  return response.data;
};
```

Just make sure the login form calls loginAdmin() instead of loginUser().

That is most likely why you're getting "Login failed".

```
this is my axios import axios from "axios"; const api = axios.create({ baseUrl:
import.meta.env.VITE_BASE_URL, headers: { "Content-Type": "application/json" }, });
api.interceptors.request.use((config) => { const token = localStorage.getItem("token"); if
(token) config.headers.Authorization = Bearer ${token}; return config; }); export default api;
this is my import api from "../axios"; // ADMIN LOGIN export const loginAdmin = async (data)
=> { const response = await api.post("/auth/login_admin", data); return response.data; }; //
ORG LOGIN export const loginOrg = async (data) => { const response = await
api.post("/auth/login_org", data); return response.data; }; // USER LOGIN export const
loginUser = async (data) => { const response = await api.post("/auth/login_user", data);
return response.data; }; and this is formimport { useState } from "react"; import { loginOrg }
from "../api/auth"; export default function LoginForm() { const [email, setEmail] =
useState(""); const [password, setPassword] = useState(""); const [loading, setLoading] =
useState(false); const [showPassword, setShowPassword] = useState(false); const
handleLogin = async (e) => { e.preventDefault(); if (!email || !password) { alert("Please fill all
fields"); return; } try { setLoading(true); // ADMIN LOGIN const response = await loginOrg({
email, password, }); console.log("LOGIN RESPONSE:", response); // SAVE TOKEN if
(response?.token) { localStorage.setItem("token", response.token); } alert("Login
successful!"); } catch (error) { console.log(error); alert( error.response?.data?.message ||
"Login failed" ); } finally { setLoading(false); } }; return ( <form onSubmit={handleLogin}> <h1
className="text-3xl font-bold mb-2"> Welcome! </h1> <p className="text-gray-500 mb-
6"> Let's discover the HR solutions in a Snap. </p> { /* EMAIL */} <div className="mb-4">
<label className="block text-sm mb-1"> Email </label> <input type="email"
placeholder="Enter your email" value={email} onChange={(e) => setEmail(e.target.value)}
className="w-full border rounded-lg p-3 focus:outline-none focus:ring-2 focus:ring-blue-
400" /> </div> { /* PASSWORD */} <div className="mb-4"> <label className="block text-sm
```

```
mb-1"> Password </label> <div className="relative"> <input type={showPassword ? "text" :
"password"} placeholder="Enter your Password" value={password} onChange={(e) =>
setPassword(e.target.value)} className="w-full border rounded-lg p-3 pr-10 focus:outline-
none focus:ring-2 focus:ring-blue-400" /> <span onClick={() =>
setShowPassword(!showPassword)} className="absolute right-3 top-3 cursor-pointer" >
</span> </div> </div> { /* REMEMBER + FORGOT */ } <div className="flex items-center
justify-between mb-6 text-sm"> <label className="flex items-center gap-2"> <input
type="checkbox" /> Remember me </label> <a href="#" className="text-blue-500"> Forgot
Password? </a> </div> { /* BUTTON */ } <div className="flex justify-end"> <button
type="submit" disabled={loading} className="w-full bg-gray-400 text-white py-3 rounded-lg
max-w-[90px]" > {loading ? "...": "Log In →"} </button> </div> </form> ); } and this is login
pageimport LeftPanel from "../components/LeftPanel"; import LoginForm from
"./components/LoginForm"; import QRLogin from "../components/QRLogin"; export default
function LoginPage() { return ( <div className="w-full min-h-screen flex flex-col lg:flex-row
bg-gray-100 p-2 gap-[15px]"> { /* LEFT */ } <div className="w-full basis-[28%] min-w-[300px]
border border-black/20 rounded-lg overflow-hidden"> <LeftPanel /> </div> { /* RIGHT */ }
<div className="w-full flex basis-[70%] items-center justify-center p-8 border border-
black/20 rounded-lg bg-white"> <div className="w-full max-w-[800px]"> <LoginForm />
<QRLogin /> </div> </div> </div> ); } why still login failed in real server api what do i need
more and where is it wrong?
```

Ah! I see exactly what's going on here. Let's break it down carefully.

You have:

1. **Axios instance** – looks correct, with Authorization header interceptor.
2. **Auth API** – separate endpoints for admin/org/user.
3. **Login form** – calls loginOrg (or loginUser/loginAdmin) directly.
4. **Login page** – just renders LoginForm.

Why “Login failed” happens

1. Wrong endpoint:

Your form calls loginOrg:

```
const response = await loginOrg({
  email,
  password,
});
```

If the server expects **admin login** and you are logging in as admin credentials, you must call loginAdmin instead.

2. CORS / Base URL mismatch:

Check that your VITE_BASE_URL in .env is pointing to the **real server URL**, e.g.:

```
VITE_BASE_URL=https://api.example.com
```

If it's still pointing to localhost or a dev server, the request will fail.

3. **Backend response format mismatch:**

Your code expects `response.data.token`. If the server sends `response.data.access_token` or another key, `if(response?.token)` will be false → "Login failed".

4. **Headers / Content-Type:**

Axios is sending `application/json`. Make sure the backend is expecting JSON and not `form-data`.

5. **Error handling:**

You're doing:

```
alert(error.response?.data?.message || "Login failed");
```

Sometimes the server responds with a different shape (like `error` instead of `message`) so you always get "Login failed".

How to fix

1 Call the correct API function

If you are logging in as **admin**, change:

```
import { loginOrg } from "../api/auth";
```

to

```
import { loginAdmin } from "../api/auth";
```

And in your form:

```
const response = await loginAdmin({ email, password });
```

2 Confirm server response

After login, add:

```
console.log("FULL LOGIN RESPONSE", response);
```

Then check:

- Does it return `token`?
- Or `access_token`?
- Or some nested object?

Update your code accordingly:

```
const token = response.token || response.access_token;
```

3 Verify Axios baseURL

.env:

```
VITE_BASE_URL=https://your-server-domain.com
```

axios.js:

```
const api = axios.create({
  baseURL: import.meta.env.VITE_BASE_URL,
  headers: { "Content-Type": "application/json" },
});
```

Make sure your frontend is using **same origin or CORS enabled** on backend.

4 Debug network

Open browser dev tools → Network → Login request → see:

- Request URL
- Request payload (JSON)
- Response body

This will tell you why it's failing. Usually, it's:

- Wrong endpoint (loginUser vs loginAdmin)
 - Wrong payload shape (server expects { username, password } not { email, password })
 - CORS / base URL
-

✓ In short:

- Ensure correct API function (loginAdmin / loginOrg / loginUser).
 - Confirm backend expects { email, password }.
 - Check server response for token key.
 - Verify VITE_BASE_URL points to production server.
-

If you want, I can **rewrite your LoginForm** so it will work correctly with your **real server API** and automatically handle admin/org/user roles without touching Redux or extra slices. This will likely fix your "Login failed" immediately.

Do you want me to do that?

```
// Lightweight auth helpers: read/set token and decode JWT payload (client-side) export
const TOKEN_KEY = 'token'; export const ORG_CODE_KEY = 'org_code'; export const
```

```

EMP_CODE_KEY = 'employee_emp_code'; export function setToken(token) { if (typeof
window === 'undefined') return; try { localStorage.setItem(TOKEN_KEY, token); } catch (e) {
// ignore } } export function getToken() { if (typeof window === 'undefined') return null; try {
return localStorage.getItem(TOKEN_KEY); } catch (e) { return null; } } export function
decodeTokenPayload(token) { try { const t = token || getToken(); if (!t || typeof t !== 'string')
return null; const base64Url = t.split('.')[1]; if (!base64Url) return null; const base64 =
base64Url.replace(/-/g, '+').replace(/_/g, '/'); const padded = base64 + '='.repeat((4 -
(base64.length % 4)) % 4); const jsonPayload = decodeURIComponent( atob(padded) .split('')
.map((c) => '%'+ ('00' + c.charCodeAt(0).toString(16)).slice(-2)) .join('') ); return
JSON.parse(jsonPayload); } catch (e) { return null; } } export function getEmpCode() { const
payload = decodeTokenPayload(); const empFromToken = payload?.emp_code ||
payload?.empCode || payload?.employee_id || null; if (empFromToken) return
empFromToken; // fallback to storage key set after create try { return typeof window !==
'undefined' ? localStorage.getItem(EMP_CODE_KEY) : null; } catch (e) { return null; } } export
function getOrgCode() { const payload = decodeTokenPayload(); const orgFromToken =
payload?.organisation_id || null; if (orgFromToken) return orgFromToken; try { return typeof
window !== 'undefined' ? localStorage.getItem(ORG_CODE_KEY) : null; } catch (e) { return
null; } } export function persistEmpCode(empCode) { try { if (typeof window !== 'undefined'
&& empCode) localStorage.setItem(EMP_CODE_KEY, empCode); } catch (e) {} } export
function persistOrgCode(orgCode) { try { if (typeof window !== 'undefined' && orgCode)
localStorage.setItem(ORG_CODE_KEY, orgCode); } catch (e) {} } // Get org_id from Redux user
object or fallback to localStorage // This handles the case where Redux user doesn't have
org_id after page refresh export function getOrgIdFromReduxOrStorage(reduxUser) { // First
check Redux user (updated by setOrgId action) if (reduxUser?.organisation_id) return
reduxUser.organisation_id; // Fallback to localStorage (persisted by setOrgId or getOrgCode)
return getOrgCode(); } this is my token.js and another file i found same when i went to
source code in inspect tool why?

```

Ah! Now this makes sense — what you're seeing in the **source code in the browser** is **your frontend's utility/helper file** for managing authentication tokens. Let me explain clearly why it exists and why it appears:

1 What token.js is doing

This file provides **client-side helpers** for:

- **Storing tokens in localStorage** (setToken)
- **Reading tokens** (getToken)
- **Decoding JWT payloads** (decodeTokenPayload)
- **Getting employee/org codes** from either the JWT or fallback localStorage
- **Persisting employee/org codes** (persistEmpCode, persistOrgCode)

Basically, it allows your frontend to know:

- Who the logged-in user is (emp_code)

- Which organization they belong to (org_code)
- Keep these values across page reloads

It's a **purely frontend helper**, no server calls, no secret keys.

2 Why you see it in the browser

When you run a React app (or Vite/Next.js/CRA), all your JS files get **bundled and sent to the browser**. That includes:

- Components (LoginForm.jsx)
- API helpers (auth.js)
- Utility files (token.js)

So when you open **Inspect → Sources**, you see token.js because it's part of the final JS bundle served to the client.

This is completely normal. It's not a secret — anyone can see your frontend code. Your secrets should only live on the **server**, never in localStorage or client JS.

3 Why your login might fail with token.js

Seeing token.js doesn't **cause** login failure. But it's related in that your login form might **expect the token in a specific key**. For example:

```
localStorage.setItem("token", response.token);
```

But the backend might send:

```
{ "access_token": "abcd1234", "user": {...} }
```

If you store response.token and it's undefined, your login fails silently.

✓ You can fix this by using token.js's setToken function:

```
import { setToken } from "../utils/token";
```

```
const response = await loginAdmin({ email, password });  
setToken(response.token || response.access_token);
```

This ensures token is **stored properly** and decodeTokenPayload() works afterward.

4 TL;DR

- token.js is **frontend token helper**. It's okay that you see it in the browser.
- It helps your frontend decode JWT, store auth info, and manage employee/org codes.
- It **does not interact with the backend directly**.

- Your login issues are likely: **wrong API endpoint**, **token field mismatch**, or **payload mismatch** — not token.js itself.
-

If you want, I can rewrite your **LoginForm** to fully **work with your token.js helpers**, automatically pick loginAdmin/loginOrg/loginUser, and store tokens safely — so it will stop showing “Login failed” on the real server.

Do you want me to do that?

how do i know what do i need in loccally to make login successful can i inspect and serach in myllive code?

Yes! You **can inspect your live site** and figure out exactly what the frontend expects in localStorage, cookies, or network requests. Here’s a clear step-by-step way to do it:

1 Open Developer Tools

- Chrome / Edge / Firefox → **Right click → Inspect**
 - Go to **Application tab** (Chrome) or **Storage tab** (Firefox)
-

2 Check Local Storage

- Look under **Local Storage → your-site.com**
 - Search for keys like:
 - token
 - auth_token
 - org_code
 - employee_emp_code
 - See what values they have (these are usually JWTs or IDs)
-

3 Check Cookies

- Still in **Application → Cookies → your-site.com**
 - Some apps store auth_token or session info in cookies instead of localStorage
-

4 Check Network Requests

- Go to the **Network tab**
- Refresh the page

- Look for login or API calls (POST /auth/login_admin etc.)
 - Click a request → **Headers** → **Request Payload**
 - This shows exactly what the frontend sends to the server (key names, values)
 - Click **Response** → **Preview** → see what server returns
 - Is the token called token or access_token?
 - Are there extra fields (user, role, org_id)?
-

5 Match it in your code

Once you know:

- The exact API endpoint the server expects
- The exact request payload keys (email vs username)
- The response token key (token vs access_token)

You can update **your frontend** to:

```
const response = await loginAdmin({ email, password });
setToken(response.token || response.access_token);
```

and it should work.

✓ Pro tip:

You can also **copy the token from localStorage** in live site and try pasting it locally. If your app loads correctly and you can access a dashboard, you know the local setup is correct.

If you want, I can make a **step-by-step method to reverse-engineer your live login** so you know exactly **what localStorage / token / payload keys** your local dev site needs to successfully log in.

Do you want me to do that?

i have got auth token and auth user after logging in in cookies

Perfect! That explains why your login is failing locally — your frontend code is currently expecting the token in **localStorage** (like token), but your **live server stores it in cookies** (auth_token and auth_user).

Here's what's happening:

1 How your code currently works

```
const token = localStorage.getItem("token");
if (token) config.headers.Authorization = `Bearer ${token}`;
```

- Your axios instance looks in **localStorage** for the token.
 - But on the live site, your server **sets a cookie** instead (auth_token).
 - So your requests **don't include a token** → backend returns "Login failed".
-

2 How to fix it

You have **two options**:

Option A — Use cookies (recommended if server sets HttpOnly cookies)

- Some servers set **HttpOnly cookies**, which are **not readable by JS**. That's more secure.
- In that case, you **don't need to manually set the token** — just let the browser send the cookie automatically with requests.

Update your Axios instance:

```
import axios from "axios";
```

```
const api = axios.create({
  baseURL: import.meta.env.VITE_BASE_URL,
  headers: { "Content-Type": "application/json" },
  withCredentials: true, //  send cookies automatically
});
```

```
export default api;
```

- withCredentials: true tells Axios to send cookies with every request.
- Remove any Authorization header logic — server already uses cookie.

Then your login form just calls:

```
const response = await loginAdmin({ email, password });
```

- You don't need localStorage.setItem("token", ...) anymore.
 - All subsequent API calls will include the cookie.
-

Option B — Store token in localStorage (if server allows)

- If your server sends token in response body (e.g., response.data.token), you can store it locally:

```
localStorage.setItem("token", response.data.token);
```

- Then axios interceptor works as is.

But your live server doesn't send token in body, it sets it in cookie — so Option A is what you want.

3 TL;DR

Since your live site uses **cookies for auth**:

1. Update Axios:

```
const api = axios.create({
  baseURL: import.meta.env.VITE_BASE_URL,
  headers: { "Content-Type": "application/json" },
  withCredentials: true, // send auth_token cookie automatically
});
```

2. Remove localStorage token logic from login form.
 3. Login will now succeed because cookies are sent automatically with API requests.
-

If you want, I can **rewrite your current LoginForm** to fully work with **cookie-based auth** so you can log in to the real server locally without touching Redux or token helpers.

Do you want me to do that?

```
. AxiosError: Network Error at XMLHttpRequest.handleError (axios.js?v=df1b496b:1777:19)
at Axios.request (axios.js?v=df1b496b:2398:41) at async loginOrg (auth.js:11:20) at async
handleLogin (LoginForm.jsx:23:24)
```

Ah! That **AxiosError: Network Error** tells us this is **not a login credential problem yet** — it's a **network-level issue**. Basically, your browser can't reach the server at all. Let's carefully break down why this might happen and how to fix it.

1 Check your baseURL

In your Axios:

```
const api = axios.create({
  baseURL: import.meta.env.VITE_BASE_URL,
  headers: { "Content-Type": "application/json" },
});
```

- Make sure VITE_BASE_URL is correctly defined in your .env file.
- It should **include protocol** (http:// or https://) and **port if necessary**, e.g.:

VITE_BASE_URL=https://api.yourdomain.com

or for local dev:

VITE_BASE_URL=http://localhost:5000

- If it's missing http:// or https://, Axios cannot connect → Network Error.
-

2 CORS / Cookies (if using cookies)

If your live server sets cookies:

- You **must** use withCredentials: true in Axios:

```
const api = axios.create({
  baseURL: import.meta.env.VITE_BASE_URL,
  headers: { "Content-Type": "application/json" },
  withCredentials: true, // ✅ important for cookies
});
```

- The server also needs to allow your frontend origin in **CORS**:

Access-Control-Allow-Origin: https://yourfrontend.com

Access-Control-Allow-Credentials: true

If this is missing, your browser **will block the request**, and Axios shows Network Error.

3 HTTPS / mixed content

- If your frontend is served over https but your API is http, the browser **blocks the request** → Network Error.
 - Solution: Make sure API URL uses the **same protocol** as frontend or enable HTTPS locally.
-

4 Firewall / VPN / local dev

- If you're testing locally against the live server: check network access.
 - Try opening your API endpoint directly in the browser (https://api.yourdomain.com/auth/login_org) to see if it responds.
-

✅ Quick test

Add a **temporary fetch test** in your LoginForm to see if your frontend can reach the server:

```
fetch(import.meta.env.VITE_BASE_URL + "/auth/login_org")
  .then(r => console.log("Server reachable:", r))
  .catch(e => console.error("Cannot reach server:", e));
```

- If this fails, the issue is **network / CORS / protocol**.
- If it works but Axios fails, probably withCredentials or headers are wrong.

Right now, your error is **not about login credentials** — it's **about network reachability or CORS**.

If you want, I can write a **full working Axios + LoginForm setup** that works with **your live server cookies**, handles CORS, and avoids Network Error.

Do you want me to do that?

```
import.meta.env = { "BASE_URL": "/", "DEV": true, "MODE": "development", "PROD": false, "SSR": false, "VITE_BASE_URL": "http://66.116.197.167:5001/api/v1/hrm" }; import axios from "/node_modules/.vite/deps/axios.js?v=203a1c7b"; const api = axios.create({ baseURL: import.meta.env.VITE_BASE_URL, headers: { "Content-Type": "application/json", }, }); //
Interceptor: Automatically add token api.interceptors.request.use( (config) => { const token = localStorage.getItem("token"); if (token) { config.headers.Authorization = Bearer ${token}; } return config; }, (error) => { return Promise.reject(error); } ); export default api; this is my axios.jsx
```

Ah! Now I see the main issues causing your **Network Error** with this Axios setup. Let's go through them carefully.

1 Your Axios baseURL is correct?

```
baseURL: import.meta.env.VITE_BASE_URL
// "http://66.116.197.167:5001/api/v1/hrm"
```

- That looks okay (it's a full URL).
 - Make sure your **frontend is running on a different port/domain** than 5001 → this is **cross-origin (CORS)** territory.
-

2 Missing withCredentials

Since your live server **sets auth cookies**, you must tell Axios to **send cookies**:

```
const api = axios.create({
  baseURL: import.meta.env.VITE_BASE_URL,
  headers: { "Content-Type": "application/json" },
  withCredentials: true, //  send cookies
});
```

Without `withCredentials: true`, cookies will **not be sent**, and the server may reject the request or block it due to CORS.

3 Authorization header

Currently, you're automatically setting:

```
const token = localStorage.getItem("token");
if (token) config.headers.Authorization = `Bearer ${token}`;
```

- On live server, your auth is **cookie-based**, not localStorage-based.
- **If the server uses cookies only**, remove this header completely:

```
api.interceptors.request.use((config) => {
  return config;
});
```

- Let withCredentials: true handle auth automatically.
-

CORS on live server

Axios **Network Error** often happens if:

- The frontend (localhost:5173) requests the backend (66.116.197.167:5001)
- The backend **doesn't allow your origin**.

The backend must return headers like:

Access-Control-Allow-Origin: http://localhost:5173

Access-Control-Allow-Credentials: true

Otherwise, the browser **blocks the request**, giving a Network Error.

Correct Axios setup for your case

```
import axios from "axios";
```

```
const api = axios.create({
  baseURL: import.meta.env.VITE_BASE_URL,
  headers: {
    "Content-Type": "application/json",
  },
  withCredentials: true, // important for cookies
});
```

```
// Remove token interceptor if backend uses cookies
```

```
// api.interceptors.request.use((config) => config);
```

```
export default api;
```

Login form

Then your login form can simply do:


```
import { loginOrg } from "../api/auth";
```

```
const response = await loginOrg({ email
```

```
no this is my server's
```

Ah! Got it — so the server is already live, and you don't control it. That means the **frontend must match exactly what the server expects**:

From your setup and the cookies you said you see:

1. **Auth is cookie-based**, not localStorage-based. 
 2. You're probably hitting **CORS issues** if withCredentials: true is not set.
 3. Sending Authorization: Bearer ... might not be needed or could break things.
-

What you need locally to make login work:

1. **Axios config** for live server:

```
import axios from "axios";
```

```
const api = axios.create({  
  baseURL: "http://66.116.197.167:5001/api/v1/hrm", // your server  
  headers: { "Content-Type": "application/json" },  
  withCredentials: true, // must send cookies  
});
```

```
export default api;
```

2. **Login form**:

```
import { loginOrg } from "../api/auth";
```

```
const handleLogin = async (e) => {  
  e.preventDefault();  
  if (!email || !password) return alert("Fill all fields");
```

```
  try {  
    setLoading(true);  
    const response = await loginOrg({ email, password });  
    console.log("Server response:", response);
```

```
    // no need to set localStorage if server sets cookie  
    alert("Login successful!");  
  } catch (err) {  
    console.log(err);  
    alert(err.response?.data?.message || "Login failed");  
  } finally {
```

```

    setLoading(false);
  }
};

```

3. Browser tools:

- Open **DevTools** → **Application** → **Cookies**. Make sure the server set `auth_token` or `auth_user`.
- Open **Network tab** → **XHR** → see request → check **Request Headers**: cookies should be sent.
- If cookies are missing → `withCredentials: true` is missing or server blocks your origin.

Key point:

You **don't need localStorage token** if server uses cookies.

The login will fail if **CORS or missing cookies** prevent the browser from sending/receiving cookies.

If you want, I can give a **small working example of LoginForm + Axios for your live server** that is guaranteed to work with cookie-based auth.

Do you want me to do that?

```

import { Box, Button, Checkbox, Flex, FormControl, FormLabel, Heading, Input, InputGroup,
InputRightElement, Link, Stack, Text, IconButton, Image, FormErrorMessage, Progress,
useToast, } from "@chakra-ui/react"; import { Navigate, Link as RouterLink } from "react-
router-dom"; import { useState, useMemo } from "react"; import { ViewIcon } from
"@chakra-ui/icons"; import hide from "../assets/hide.png"; import { useForm } from
"react-hook-form"; import { useDispatch, useSelector } from "react-redux"; import {
loginUserThunk } from "../slices/authSlice"; import { useNavigate } from "react-router-
dom"; import { LuEye } from "react-icons/lu"; import { LuEyeOff } from "react-icons/lu";
import Cookies from "js-cookie"; const EMAIL_REGEX = /^[a-zA-Z0-9_+]+@[a-zA-Z0-9-
]+\.[a-zA-Z0-9-]+$/; export default function LoginForm({ onSuccess } = {}) { const
[showPassword, setShowPassword] = useState(false); const toast = useToast(); const
dispatch = useDispatch(); const navigate = useNavigate(); const { loading, error } =
useSelector((state) => state.auth); const token = Cookies.get("auth_token"); const inputStyle
= { fontFamily: "Poppins", fontSize: "15px", _placeholder: { color: "gray.400", fontFamily:
"Poppins", fontSize: "14px", }, h: "55px", borderWidth: "1.5px", borderColor: "#D0D5DD",
borderRadius: "8px", }, // if (token) { // return <Navigate to="/dashboard" replace />; // } //
react-hook-form config const { register, handleSubmit, watch, reset, formState: { errors,
isSubmitting, isValid }, } = useForm({ mode: "onChange", criteriaMode: "all", defaultValues: {
email: "", password: "", remember: false, }, }); // watch input values const emailValue =
watch("email", ""); const passwordValue = watch("password", ""); // helper checks for "field
is valid" used for progress calculation const isEmailValid = useMemo( () =>
EMAIL_REGEX.test(emailValue), [emailValue] ); const isPasswordValidForProgress =
useMemo( () => passwordValue && passwordValue.length >= 8, [passwordValue] ); //

```

```

compute progress percent (2 inputs: email + password) const progressPct = useMemo(() => {
const total = 2; const completed = (isEmailValid ? 1 : 0) + (isPasswordValidForProgress ? 1 :
0); return Math.round((completed / total) * 100); }, [isEmailValid,
isPasswordValidForProgress]); // API integration and show data in console const onSubmit =
async (data) => { try { const loginData = { email: data.email, password: data.password }; const
result = await dispatch(loginUserThunk(loginData)); if (result.meta.requestStatus ===
"fulfilled") { toast({ title: "Logged in", description: "You have successfully logged in.", status:
"success", duration: 2500, isClosable: true, }); reset(); const token = result.payload?.token ||
result.payload?.data?.token || result.payload; // decode JWT payload (safe base64url
decode) const decodeJwtPayload = (t) => { try { if (!t || typeof t !== "string") return null;
const base64Url = t.split(".")[1]; if (!base64Url) return null; const base64 =
base64Url.replace(/-/g, "+").replace(/_/g, "/"); const padded = base64 + "=".repeat((4 -
(base64.length % 4)) % 4); const jsonPayload = decodeURIComponent( atob(padded)
.split("").map( (c) => "%" + ("00" + c.charCodeAt(0).toString(16)).slice(-2) ) .join("")); return
JSON.parse(jsonPayload); } catch (e) { return null; } }; // store token if present if (token &&
typeof token === "string") { try { localStorage.setItem("token", token); } catch (e) {
console.error("Failed to store token in localStorage:", e); } const decoded =
decodeJwtPayload(token); const isOrgPresent = decoded?.is_org_present ??
decoded?.isOrgPresent ?? false; const isAdmin = decoded?.is_admin ?? decoded?.isAdmin
?? false; if (isOrgPresent) { navigate("/dashboard", { replace: true }); } else { // Only admins
should be allowed to create an organization profile. if (isAdmin) {
import("../pages/Dashboard"); import("../layouts/AppLayout");
import("../pages/OrganizationRegister"); navigate("/orgRegister", { replace: true }); } else {
// Non-admin employees should be redirected to dashboard and informed
navigate("/dashboard", { replace: true }); } } } else { navigate("/orgRegister"); } if
(onSubmitSuccess) onSubmitSuccess(result.payload); // Optionally, redirect or store token
here } else { toast({ title: "Login failed", description: result.payload?.message || "Invalid
credentials.", status: "error", duration: 3500, isClosable: true, }); } } catch (err) { toast({ title:
"Login failed", description: "Something went wrong. Please try again.", status: "error",
duration: 3500, isClosable: true, }); } }; return ( <Flex flexDirection="column" justify="center"
align="flex-start" w="100%" fontFamily={"Poppins"} > { /* Previously there was a progress
bar here but we are currently not rendering it.. */ <Box bg="white" mt="25px" px={{ base:
6, md: 75 }} w="100%" borderRadius="8px" boxSizing="border-box" > { /* Use a real <form>
so Enter submits */ <form onSubmit={handleSubmit(onSubmit)}> <Stack spacing={4}>
<Box> <Heading // fontSize={{ "20px", "22px", "24px" }} fontSize="30px" mb={1}
fontFamily={"Poppins", sans-serif} fontWeight={700} color="#000000" lineHeight="40px" >
Welcome! </Heading> <Text color="#757575" fontSize="sm" fontFamily={"Poppins", sans-
serif" > Let's discover the HR solutions in a Snap. </Text> </Box> { /* Email */} <FormControl
id="email" isValid={!errors.email} mt={0}> <FormLabel fontFamily={"Poppins", sans-serif"
fontWeight="500" fontSize="14px" color="#000000" > Email </FormLabel> <Input
type="email" placeholder="Enter your email" borderRadius={4} fontSize="14px"
{...register("email", { required: "Email is required", pattern: { value: EMAIL_REGEX, message:
"Enter a valid email address", }, })} aria-invalid={errors.email ? "true" : "false"} {...inputStyle}
/> <FormErrorMessage fontSize="12px" fontFamily={"poppins"} fontWeight={"500"}>
{errors.email?.message} </FormErrorMessage> </FormControl> { /* Password */}
<FormControl id="password" isValid={!errors.password} mt={3}> <FormLabel
fontFamily={"Poppins", sans-serif" fontWeight="500" fontSize="14px" color="#000000" >

```

```

Password </FormLabel> <InputGroup> <Input type={showPassword ? "text" : "password"}
placeholder="Enter your Password" borderRadius={4} fontSize="14px"
{...register("password", { required: "Password is required", })} aria-invalid={errors.password ?
"true" : "false"} {...inputStyle} /> <InputRightElement h="full"> <IconButton type="button" //
prevent this icon from submitting the form aria-label={ showPassword ? "Hide password" :
"Show password" } icon={showPassword ? <LuEyeOff /> : <LuEye />} onClick={() =>
setShowPassword((s) => !s)} variant="ghost" size="sm" _hover={{ bg: "transparent" }}
_active={{ bg: "transparent" }} /> </InputRightElement> </InputGroup> { /* Password hint /
errors */ } {errors.password?.types ? ( <Box mt={2}>
{Object.values(errors.password.types).map((msg, idx) => ( <Text key={idx} color="red.500"
fontSize="12px" fontFamily={'poppins'} fontWeight={'500'}> {msg} </Text> ))) </Box> ) : (
<FormErrorMessage fontSize={'12px'} fontFamily={'poppins'}
fontWeight={'500'}>{errors.password?.message}</FormErrorMessage> )} </FormControl>
{ /* Remember + Forgot */} <Flex justify="space-between" align="center"> <Checkbox
colorScheme="blue" {...register("remember")} sx={{ ".chakra-checkbox__control": { w:
"20px", h: "20px", }, }} > <Text fontSize="13px" fontFamily="Poppins", sans-serif"
fontWeight="500" color="#274167"> Remember me </Text> </Checkbox> <Link
as={RouterLink} to="/forgot-password" color="#274167" fontSize="13px"
fontFamily="Poppins", sans-serif" fontWeight="500" textDecoration="underline"> Forgot
Password? </Link> </Flex> { /* Button */} <Box display="flex" justifyContent="flex-end">
<Button type="submit" bg="#274167" color="white" fontFamily="Poppins", sans-serif"
fontWeight="500" fontSize="14px" _active={{ bg: "#1f3143" }} _hover={{ bg: "#1f3143" }}
rightIcon={<span>→</span>} isLoading={loading} isDisabled={!isValid || loading} aria-
disabled={!isValid || loading} mt={6} mb={-4}> Log In </Button> </Box> </Stack> </form>
</Box> </Flex> ); } this is my servers login form in server and this is my local import {
useState } from "react"; import { loginAdmin } from "../api/auth"; export default function
LoginForm() { const [email, setEmail] = useState(""); const [password, setPassword] =
useState(""); const [loading, setLoading] = useState(false); const [showPassword,
setShowPassword] = useState(false); const handleLogin = async (e) => { e.preventDefault(); if
(!email || !password) { alert("Please fill all fields"); return; } try { setLoading(true); // ADMIN
LOGIN const response = await loginAdmin({ email, password, }); console.log("LOGIN
RESPONSE:", response); // SAVE TOKEN if (response?.token) { localStorage.setItem("token",
response.token); } alert("Login successful!"); } catch (error) { console.log(error); alert(
error.response?.data?.message || "Login failed" ); } finally { setLoading(false); } }; return (
<form onSubmit={handleLogin}> <h1 className="text-3xl font-bold mb-2"> Welcome!
</h1> <p className="text-gray-500 mb-6"> Let's discover the HR solutions in a Snap. </p>
{ /* EMAIL */} <div className="mb-4"> <label className="block text-sm mb-1"> Email
</label> <input type="email" placeholder="Enter your email" value={email} onChange={(e)
=> setEmail(e.target.value)} className="w-full border rounded-lg p-3 focus:outline-none
focus:ring-2 focus:ring-blue-400" /> </div> { /* PASSWORD */} <div className="mb-4">
<label className="block text-sm mb-1"> Password </label> <div className="relative">
<input type={showPassword ? "text" : "password"} placeholder="Enter your Password"
value={password} onChange={(e) => setPassword(e.target.value)} className="w-full border
rounded-lg p-3 pr-10 focus:outline-none focus:ring-2 focus:ring-blue-400" /> <span
onClick={() => setShowPassword(!showPassword)} className="absolute right-3 top-3
cursor-pointer" >  </span> </div> </div> { /* REMEMBER + FORGOT */} <div
className="flex items-center justify-between mb-6 text-sm"> <label className="flex items-

```

```
center gap-2"> <input type="checkbox" /> Remember me </label> <a href="#"
className="text-blue-500"> Forgot Password? </a> </div> { /* BUTTON */ } <div
className="flex justify-end"> <button type="submit" disabled={loading} className="w-full
bg-gray-400 text-white py-3 rounded-lg max-w-[90px]" > {loading ? "...": "Log In →"}
</button> </div> </form> ); } whats problem
```

Ah, now I can see the full picture. Thanks for sharing both forms. The problem is **why your local login fails while the server login works**. Let's go step by step.

Key Differences Between Server Form and Your Local Form

1. Server uses Redux Thunks + Cookies

- The server login form dispatches `loginUserThunk`.
- The token is set in **cookies (auth_token)**, not `localStorage`.
- `withCredentials: true` is probably enabled in Axios on the server.
- Redirections depend on **decoded token info** (`is_org_present`, `is_admin`).

2. Your local form uses direct Axios call

- You call `loginAdmin` which posts to `/auth/login_admin`.
 - You store token in `localStorage`.
 - **No `withCredentials`**, so cookies won't be stored by the browser.
-

Likely Problems Causing Login Failure

1. Server expects cookies, not Authorization header

- Your Axios interceptor adds `Authorization: Bearer ...` using `localStorage` token.
- Server might ignore it or reject the request.
- If the server is cookie-based auth, your token storage in `localStorage` is irrelevant.

2. CORS issue

- If you call `http://66.116.197.167:5001` from `localhost`, the browser may block cookies unless `withCredentials: true` is set on Axios.

3. Wrong endpoint or payload

- You call `loginAdmin`, but maybe for normal user/org login you should call `/auth/login_user` or `/auth/login_org`.
 - Payload may require `{ email, password }` but server may also expect `remember` or other fields.
-

How to Fix Locally

1 Update Axios to send cookies

```
import axios from "axios";

const api = axios.create({
  baseURL: "http://66.116.197.167:5001/api/v1/hrm",
  headers: {
    "Content-Type": "application/json",
  },
  withCredentials: true, // <--- important for cookie auth
});

export default api;
```

2 Update login function to just send credentials

```
import { loginOrg } from "../api/auth";

const handleLogin = async (e) => {
  e.preventDefault();
  if (!email || !password) return alert("Fill all fields");

  try {
    setLoading(true);

    // send email + password, cookies will be handled by browser
    const response = await loginOrg({ email, password });
    console.log("Server response:", response);

    alert("Login successful!");
    // Redirect manually if needed
    window.location.href = "/dashboard";

  } catch (err) {
    console.error(err);
    alert(err.response?.data?.message || "Login failed");
  } finally {
    setLoading(false);
  }
};
```

3 Do NOT manually set token in localStorage if server sets cookie

- Remove `localStorage.setItem("token", response.token)` — the server cookie is enough.

4 Optional: inspect network

- Open DevTools → Network → XHR → login request.
- Check Set-Cookie header in response.
- Ensure subsequent requests send cookie.

✅ Summary

- Your local login fails because **you are not sending credentials/cookies properly**.
- The server login works because it handles **cookies + Redux thunk**, not localStorage.
- **Fix:** Axios withCredentials: true, remove localStorage, make sure payload matches server.

If you want, I can rewrite **your entire local login form** to **mirror the server login exactly** and make it work with the live server in **<20 lines**.

Do you want me to do that?

yes please

Perfect! Let's rewrite your **local login form** so it works exactly like your server login. This version will:

- Send credentials properly (email + password).
- Use **Axios with withCredentials: true** to allow cookies.
- Handle login success and redirect to dashboard.
- Remove localStorage usage since server sets cookies.

Here's a **minimal working version**:

```
import { useState } from "react";
import api from "../api/axios"; // Axios instance with withCredentials: true
import { useNavigate } from "react-router-dom";
```

```
export default function LoginForm() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [loading, setLoading] = useState(false);
  const navigate = useNavigate();
```

```
  const handleLogin = async (e) => {
    e.preventDefault();
    if (!email || !password) return alert("Please fill all fields");
```

```
    try {
      setLoading(true);
```

```

// Send login request
const response = await api.post("/auth/login_org", { email, password });
console.log("Login response:", response.data);

// Server sets cookie, no need for localStorage
alert("Login successful!");

// Redirect to dashboard
navigate("/dashboard", { replace: true });
} catch (error) {
  console.error(error);
  alert(error.response?.data?.message || "Login failed");
} finally {
  setLoading(false);
}
};

return (
  <form onSubmit={handleLogin} className="w-full max-w-md mx-auto p-4">
    <h1 className="text-2xl font-bold mb-4">Welcome!</h1>

    <input
      type="email"
      placeholder="Email"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
      className="w-full mb-3 p-2 border rounded"
    />

    <input
      type="password"
      placeholder="Password"
      value={password}
      onChange={(e) => setPassword(e.target.value)}
      className="w-full mb-3 p-2 border rounded"
    />

    <button
      type="submit"
      disabled={loading}
      className="w-full bg-blue-600 text-white py-2 rounded"
    >
      {loading ? "Logging in..." : "Log In"}
    </button>
  </form>

```



```
};  
}
```

Axios (api/axios.js)

Make sure your Axios instance looks like this:

```
import axios from "axios";  
  
const api = axios.create({  
  baseURL: "http://66.116.197.167:5001/api/v1/hrm",  
  headers: { "Content-Type": "application/json" },  
  withCredentials: true, // <-- important for cookie auth  
});  
  
export default api;
```

✅ Why this works

- The server sends Set-Cookie on successful login.
 - withCredentials: true ensures browser stores and sends the cookie.
 - No need for localStorage.
 - Next API calls to the server will automatically include the cookie.
-

If you want, I can also **add “remember me” and show/hide password functionality** exactly like your server form, so it fully matches the live UI.

Do you want me to do that?

```
// import axios from "axios"; // const api = axios.create({ // baseURL:  
"http://66.116.197.167:5001/api/v1/hrm", // headers: { // "Content-Type":  
"application/json", // }, // }); // // optional: attach token globally //  
api.interceptors.request.use((config) => { // const token = localStorage.getItem("token"); // if  
(token) { // config.headers.Authorization = Bearer ${token}; // } // return config; // }); //  
export default api; import axios from "axios"; const api = axios.create({ baseURL:  
import.meta.env.VITE_BASE_URL, headers: { "Content-Type": "application/json" },  
withCredentials: true, // send auth_token cookie automatically }); export default api; this is  
,my code now and i do have withcredentials true just that i donot have ciookies provision  
and redux thunk
```

Ah! That clears things up. Your **Axios setup is fine** — withCredentials: true will automatically send cookies.

The key problems you have right now are:

1. **Server sets cookies** (auth_token and auth_user) on login. You're not using Redux, so you don't automatically update the client state.
 2. **Your local form stores tokens in localStorage**, but your server expects cookie-based auth. That's why login works in live code but fails locally.
 3. **Redux Thunk isn't used**, so you don't have loginUserThunk to manage async state or dispatch actions. That's okay — you can skip Redux entirely if you just want a working login.
-

How to make login work locally without Redux/Thunks

1. Remove any localStorage.setItem("token") — your server handles auth via cookies.
2. Make sure your Axios request uses withCredentials: true (which you already do).
3. Handle successful login by **redirecting**, because the cookie will automatically authorize further requests.

Here's a **minimal working example**:

```
import { useState } from "react";
import api from "../api/axios";
import { useNavigate } from "react-router-dom";

export default function LoginForm() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [loading, setLoading] = useState(false);
  const navigate = useNavigate();

  const handleLogin = async (e) => {
    e.preventDefault();
    if (!email || !password) return alert("Please fill all fields");

    try {
      setLoading(true);

      const response = await api.post("/auth/login_org", { email, password });
      console.log("Server response:", response.data);

      alert("Login successful!");
      navigate("/dashboard"); // redirect, cookies handle auth
    } catch (error) {
      console.error(error);
      alert(error.response?.data?.message || "Login failed");
    } finally {
      setLoading(false);
    }
  }
}
```

```

};

return (
  <form onSubmit={handleLogin}>
    <input
      type="email"
      placeholder="Email"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
    />
    <input
      type="password"
      placeholder="Password"
      value={password}
      onChange={(e) => setPassword(e.target.value)}
    />
    <button type="submit" disabled={loading}>
      {loading ? "Logging in..." : "Log In"}
    </button>
  </form>
);
}

```

✓ Key points

- **No need for localStorage token** because the server sends a cookie.
 - **All further API calls** should use the same Axios instance (withCredentials: true) to include the cookie.
 - If you want **Redux/Thunk**, you can wrap this request in a thunk to manage loading and error state globally, but it's optional.
-

If you want, I can rewrite your **full Chakra UI login form** to work with **cookie-based login**, exactly like your server code — including show/hide password, remember me, validation, and proper redirects.

Do you want me to do that?

if i use the above given code should i delete the auth.js file

Not necessarily — it depends on how you want to organize your code. Here's the breakdown:

Current auth.js

```
import api from "../axios";
```

```
export const loginAdmin = async (data) => {
  const response = await api.post("/auth/login_admin", data);
  return response.data;
};
```

- It **wraps your Axios requests** and returns response.data.
- Useful if you plan to **reuse login calls** in multiple components.

Using my minimal example

- You call api.post("/auth/login_org", ...) **directly in the component**.
- You **don't need auth.js**, unless you want to keep your API calls separate for clarity.

Recommendation

- **Keep auth.js** if you want a clean separation of API logic from UI.
- Your login form would then look like:

```
import { loginOrg } from "../api/auth";
```

```
const response = await loginOrg({ email, password });
```

- **Otherwise**, you can delete auth.js and call Axios directly in the component — both approaches work.

💡 **Tip:** For cookie-based auth, you **don't need to handle tokens in localStorage**, so the auth.js file becomes simpler: just export functions that call Axios.

If you want, I can **rewrite your auth.js and your login form** so they work cleanly with cookies, without storing any tokens. This way you can keep your code modular. Do you want me to do that?

okay the ting is i have the correct credentials and these are the details that are coming {
"_id": "694e7a36c3e6287f4b0e2a65", "employee_id": "EMP000056", "first_name":
"Kumkum", "middle_name": "", "last_name": "Dutta", "full_name": "Kumkum Dutta",
"email": "kumkum.dutta@delostylestudio.com", "password":
"\$2b\$10\$FC0AX1DAdgiBYeqz./Pdce7Zf9e71vxFf2DG/j5klNRjcVz8z/cOi", "personal_email":
"", "mobile": "9382068271", "dob": "1970-01-01", "gender": "Female", "marital_status":
"6923f7ce7314cf8bf085a95d", "passport_number": "", "blood_group": "", "profile_photo":
"", "is_active": true, "created_at": "2025-11-14T04:59:22.632Z", "updated_at": "2026-01-
06T09:55:52.863Z", "role": "69148030869483846f611108", "organisation_id":
"HRMS000017", "nationality": "", "password_length": 12, "is_admin": true,
"is_org_present": true, "org_id": "HRMS000017", "shift": "69491c2328c752ab88d3928c",
"punch_status": 2, "iat": 1778152289, "exp": 1778757089 } as token value gets stored in

cookies and local storage when logged in so tell me how do i structure my login form dynamically so that i can login locally and go to server's dashboard

Ah! Perfect — now I understand exactly what you have. You have a **server that sets cookies** (auth_token) and returns a **JWT payload with user details**, and you want your **local login form** to work with that setup, dynamically handling the login without storing passwords locally. Let's structure it clearly.

1 API Helper (auth.js)

You can keep this file for clean separation:

```
import api from "../axios"; // your axios instance with withCredentials: true
```

```
export const loginUser = async (data) => {  
  // data = { email, password }  
  const response = await api.post("/auth/login_user", data);  
  return response.data; // this is the payload you showed  
};
```

✓ Notes:

- No need to manually store token; your server sets the cookie automatically.
 - Only return the user data from response.data.
-

2 Login Form

Here's a simple dynamic form that handles login, stores minimal data, and navigates correctly:

```
import { useState } from "react";  
import { useNavigate } from "react-router-dom";  
import { loginUser } from "../api/auth";  
  
export default function LoginForm() {  
  const [email, setEmail] = useState("");  
  const [password, setPassword] = useState("");  
  const [loading, setLoading] = useState(false);  
  const [showPassword, setShowPassword] = useState(false);  
  
  const navigate = useNavigate();  
  
  const handleLogin = async (e) => {  
    e.preventDefault();  
    if (!email || !password) {  
      alert("Please fill all fields");  
      return;  
    }  
    // ... login logic ...  
  }  
}
```

```

}

try {
  setLoading(true);

  // Call the server login
  const user = await loginUser({ email, password });
  console.log("Logged in user:", user);

  // Store basic user info if needed locally
  localStorage.setItem("user", JSON.stringify({
    id: user._id,
    full_name: user.full_name,
    email: user.email,
    is_admin: user.is_admin,
    org_id: user.org_id,
  }));

  // Navigate dynamically based on server payload
  if (user.is_org_present) {
    navigate("/dashboard"); // org present → dashboard
  } else if (user.is_admin) {
    navigate("/orgRegister"); // admin, no org → org creation page
  } else {
    navigate("/dashboard"); // regular employee
  }

} catch (err) {
  console.error(err);
  alert(
    err.response?.data?.message || "Login failed, check your credentials"
  );
} finally {
  setLoading(false);
}
};

return (
  <form onSubmit={handleLogin}>
    <h1>Welcome!</h1>

    <div>
      <label>Email</label>
      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}

```

```

        placeholder="Enter your email"
      />
    </div>

    <div>
      <label>Password</label>
      <input
        type={showPassword ? "text" : "password"}
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        placeholder="Enter your password"
      />
      <button type="button" onClick={() => setShowPassword(!showPassword)}>
        {showPassword ? "Hide" : "Show"}
      </button>
    </div>

    <button type="submit" disabled={loading}>
      {loading ? "Logging in..." : "Log In"}
    </button>
  </form>
);
}

```

3 Axios Configuration

Your Axios is already almost correct. Make sure **withCredentials** is set:

```

import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_BASE_URL,
  headers: { "Content-Type": "application/json" },
  withCredentials: true, // ✅ important for cookies
});

export default api;

```

4 Key Points

1. **Cookies:** No need to manually store JWT; server does it automatically. Just set `withCredentials: true`.
2. **Local Storage:** Only store **minimal info** like user id, name, org id — do **not store passwords**.
3. **Dynamic navigation:** Use `is_org_present` and `is_admin` from JWT to redirect.

4. **Local login:** You can test locally with same credentials, just ensure VITE_BASE_URL points to the real server (e.g., <http://66.116.197.167:5001/api/v1/hrm>).

If you want, I can **rewrite your full local login page** exactly like the server's one, fully dynamic with Chakra UI and proper cookie handling, so it behaves identically to your server's login.

Do you want me to do that?

i will provide the axios and auth file of my server

Perfect! Go ahead and share your **server's axios and auth files**, and I can craft a **matching local login form** that works with your server API, cookies, and dynamic redirects exactly like your live setup.

```
import api from "/src/api/axios.js"; export const registerAdmin = async (data) => { try { const response = await api.post("/auth/register_admin", data); return response.data; } catch (error) { console.log(error); throw error; } }; export const registerOrg = async (data) => { try { const response = await api.post("/auth/register_org", data); return response.data; } catch (error) { console.log(error); throw error; } }; export const loginUser = async (data) => { const response = await api.post("/auth/login_user", data); return response.data; } ;
```

```
import.meta.env = {"BASE_URL": "/", "DEV": true, "MODE": "development", "PROD": false, "SSR": false, "VITE_BASE_URL": "http://66.116.197.167:5001/api/v1/hrm"}; import axios from "/node_modules/.vite/deps/axios.js?v=203a1c7b"; const api = axios.create({ baseURL: import.meta.env.VITE_BASE_URL, headers: { "Content-Type": "application/json", }, }); // Interceptor: Automatically add token api.interceptors.request.use( (config) => { const token = localStorage.getItem("token"); if (token) { config.headers.Authorization = Bearer ${token}; } return config; }, (error) => { return Promise.reject(error); } ); export default api;
```

Thanks! Now that I

```
i have provided the auth and axios file and now i am going to provide the loginform import { Box, Button, Checkbox, Flex, FormControl, FormLabel, Heading, Input, InputGroup, InputRightElement, Link, Stack, Text, IconButton, Image, FormErrorMessage, Progress, useToast, } from "@chakra-ui/react"; import { Navigate, Link as RouterLink } from "react-router-dom"; import { useState, useMemo } from "react"; import { ViewIcon } from "@chakra-ui/icons"; import hide from "../assets/hide.png"; import { useForm } from "react-hook-form"; import { useDispatch, useSelector } from "react-redux"; import { loginUserThunk } from "../slices/authSlice"; import { useNavigate } from "react-router-dom"; import { LuEye } from "react-icons/lu"; import { LuEyeOff } from "react-icons/lu"; import Cookies from "js-cookie"; const EMAIL_REGEX = /^[a-zA-Z0-9_+]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+\.$/; export default function LoginForm({ onSubmitSuccess } = {}) { const [showPassword, setShowPassword] = useState(false); const toast = useToast(); const dispatch = useDispatch(); const navigate = useNavigate(); const { loading, error } = useSelector((state) => state.auth); const token = Cookies.get("auth_token"); const inputStyle = { fontFamily: "Poppins", fontSize: "15px", _placeholder: { color: "gray.400", fontFamily: "Poppins", fontSize: "14px", }, h: "55px", borderWidth: "1.5px", borderColor: "#D0D5DD", borderRadius: "8px", }; // if (token) { // return <Navigate to="/dashboard" replace />; // } //
```



```

react-hook-form config const { register, handleSubmit, watch, reset, formState: { errors,
isSubmitting, isValid }, } = useForm({ mode: "onChange", criteriaMode: "all", defaultValues: {
email: "", password: "", remember: false, }, }); // watch input values const emailValue =
watch("email", ""); const passwordValue = watch("password", ""); // helper checks for "field
is valid" used for progress calculation const isEmailValid = useMemo( () =>
EMAIL_REGEX.test(emailValue), [emailValue] ); const isPasswordValidForProgress =
useMemo( () => passwordValue && passwordValue.length >= 8, [passwordValue] ); //
compute progress percent (2 inputs: email + password) const progressPct = useMemo(() => {
const total = 2; const completed = (isEmailValid ? 1 : 0) + (isPasswordValidForProgress ? 1 :
0); return Math.round((completed / total) * 100); }, [isEmailValid,
isPasswordValidForProgress]); // API integration and show data in console const onSubmit =
async (data) => { try { const loginData = { email: data.email, password: data.password }; const
result = await dispatch(loginUserThunk(loginData)); if (result.meta.requestStatus ===
"fulfilled") { toast({ title: "Logged in", description: "You have successfully logged in.", status:
"success", duration: 2500, isClosable: true, }); reset(); const token = result.payload?.token ||
result.payload?.data?.token || result.payload; // decode JWT payload (safe base64url
decode) const decodeJwtPayload = (t) => { try { if (!t || typeof t !== "string") return null;
const base64Url = t.split(".")[1]; if (!base64Url) return null; const base64 =
base64Url.replace(/-/g, "+").replace(/_/g, "/"); const padded = base64 + "=".repeat((4 -
(base64.length % 4)) % 4); const jsonPayload = decodeURIComponent( atob(padded)
.split("").map( (c) => "%" + ("00" + c.charCodeAt(0).toString(16)).slice(-2) ) .join("")); return
JSON.parse(jsonPayload); } catch (e) { return null; } }; // store token if present if (token &&
typeof token === "string") { try { localStorage.setItem("token", token); } catch (e) {
console.error("Failed to store token in localStorage:", e); } } const decoded =
decodeJwtPayload(token); const isOrgPresent = decoded?.is_org_present ??
decoded?.isOrgPresent ?? false; const isAdmin = decoded?.is_admin ?? decoded?.isAdmin
?? false; if (isOrgPresent) { navigate("/dashboard", { replace: true }); } else { // Only admins
should be allowed to create an organization profile. if (isAdmin) {
import("../pages/Dashboard"); import("../layouts/AppLayout");
import("../pages/OrganizationRegister"); navigate("/orgRegister", { replace: true }); } else {
// Non-admin employees should be redirected to dashboard and informed
navigate("/dashboard", { replace: true }); } } } else { navigate("/orgRegister"); } if
(onSubmitSuccess) onSubmitSuccess(result.payload); // Optionally, redirect or store token
here } else { toast({ title: "Login failed", description: result.payload?.message || "Invalid
credentials.", status: "error", duration: 3500, isClosable: true, }); } } catch (err) { toast({ title:
>Login failed", description: "Something went wrong. Please try again.", status: "error",
duration: 3500, isClosable: true, }); } }; return ( <Flex flexDirection="column" justify="center"
align="flex-start" w="100%" fontFamily={"Poppins"} > { /* Previously there was a progress
bar here but we are currently not rendering it.. */ <Box bg="white" mt="25px" px={{ base:
6, md: 75 }} w="100%" borderRadius="8px" boxSizing="border-box" > { /* Use a real <form>
so Enter submits */ <form onSubmit={handleSubmit(onSubmit)}> <Stack spacing={4}>
<Box> <Heading // fontSize={["20px", "22px", "24px"]} fontSize="30px" mb={1}
fontFamily={"Poppins", sans-serif} fontWeight={700} color="#000000" lineHeight="40px" >
Welcome! </Heading> <Text color="#757575" fontSize="sm" fontFamily={"Poppins", sans-
serif" > Let's discover the HR solutions in a Snap. </Text> </Box> { /* Email */ <FormControl
id="email" isValid={!errors.email} mt={0}> <FormLabel fontFamily={"Poppins", sans-serif"
fontWeight="500" fontSize="14px" color="#000000" > Email </FormLabel> <Input

```

```

type="email" placeholder="Enter your email" borderRadius={4} fontSize="14px"
{...register("email", { required: "Email is required", pattern: { value: EMAIL_REGEX, message:
"Enter a valid email address", }, })} aria-invalid={errors.email ? "true" : "false"} {...inputStyle}
/> <FormErrorMessage fontSize="12px" fontFamily={'poppins'} fontWeight={'500'}>
{errors.email?.message} </FormErrorMessage> </FormControl> { /* Password */
<FormControl id="password" isValid={!errors.password} mt={3}> <FormLabel
fontFamily="'Poppins', sans-serif" fontWeight="500" fontSize="14px" color="#000000" >
Password </FormLabel> <InputGroup> <Input type={showPassword ? "text" : "password"}
placeholder="Enter your Password" borderRadius={4} fontSize="14px"
{...register("password", { required: "Password is required", })} aria-invalid={errors.password ?
"true" : "false"} {...inputStyle} /> <InputRightElement h="full"> <IconButton type="button" //
prevent this icon from submitting the form aria-label={ showPassword ? "Hide password" :
"Show password" } icon={showPassword ? <LuEyeOff /> : <LuEye />} onClick={() =>
setShowPassword((s) => !s)} variant="ghost" size="sm" _hover={{ bg: "transparent" }}
_active={{ bg: "transparent" }} /> </InputRightElement> </InputGroup> { /* Password hint /
errors */ } {errors.password?.types ? ( <Box mt={2}>
{Object.values(errors.password.types).map((msg, idx) => ( <Text key={idx} color="red.500"
fontSize="12px" fontFamily={'poppins'} fontWeight={'500'}> {msg} </Text> ))} </Box> ) : (
<FormErrorMessage fontSize={'12px'} fontFamily={'poppins'}
fontWeight={'500'}>{errors.password?.message}</FormErrorMessage> )} </FormControl>
{ /* Remember + Forgot */ <Flex justify="space-between" align="center"> <Checkbox
colorScheme="blue" {...register("remember")} sx={{ ".chakra-checkbox__control": { w:
"20px", h: "20px", }, }}> <Text fontSize="13px" fontFamily="'Poppins', sans-serif"
fontWeight="500" color="#274167"> Remember me </Text> </Checkbox> <Link
as={RouterLink} to="/forgot-password" color="#274167" fontSize="13px"
fontFamily="'Poppins', sans-serif" fontWeight="500" textDecoration="underline"> Forgot
Password? </Link> </Flex> { /* Button */ <Box display="flex" justifyContent="flex-end">
<Button type="submit" bg="#274167" color="white" fontFamily="'Poppins', sans-serif"
fontWeight="500" fontSize="14px" _active={{ bg: "#1f3143" }} _hover={{ bg: "#1f3143" }}
rightIcon={<span>→</span>} isLoading={loading} isDisabled={!isValid || loading} aria-
disabled={!isValid || loading} mt={6} mb={-4}> Log In </Button> </Box> </Stack> </form>
</Box> </Flex> ); } this is server's now make a dupe of this project so that i can login in local
form and go to server;s dashboard tell me if you want something

```

Perfect! I now have a complete picture: your server's login form uses **Chakra UI, React Hook Form, JWT stored in localStorage**, and conditional navigation based on `is_org_present` and `is_admin`.

To replicate this **locally**, all we need to do is:

1. Keep your **local login form structurally identical** to the server's.
2. Replace the **Redux loginUserThunk** call with a direct call to your `auth.js` function (`loginUser`) that hits the server API.
3. Handle **token storage, decoding, and navigation** exactly like your server form.
4. Keep all Chakra UI styling, form validation, progress calculation, show/hide password, etc.

Here's a **full local version of your login form** that will work with your server API:

```
import {
  Box,
  Button,
  Checkbox,
  Flex,
  FormControl,
  FormLabel,
  Heading,
  Input,
  InputGroup,
  InputRightElement,
  Link,
  Stack,
  Text,
  IconButton,
  FormErrorMessage,
  useToast,
} from "@chakra-ui/react";
import { useState, useMemo } from "react";
import { useForm } from "react-hook-form";
import { LuEye, LuEyeOff } from "react-icons/lu";
import { useNavigate } from "react-router-dom";
import { loginUser } from "../api/auth"; // your local auth.js function

const EMAIL_REGEX = /^[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+\$/;

export default function LocalLoginForm({ onSubmitSuccess } = {}) {
  const [showPassword, setShowPassword] = useState(false);
  const [loading, setLoading] = useState(false);
  const toast = useToast();
  const navigate = useNavigate();

  const inputStyle = {
    fontFamily: "Poppins",
    fontSize: "15px",
    _placeholder: { color: "gray.400", fontSize: "14px" },
    h: "55px",
    borderWidth: "1.5px",
    borderColor: "#D0D5DD",
    borderRadius: "8px",
  };

  const {
    register,
  }
```

```

    handleSubmit,
    watch,
    reset,
    formState: { errors, isValid },
  } = useForm({
    mode: "onChange",
    defaultValues: { email: "", password: "", remember: false },
  });

const emailValue = watch("email", "");
const passwordValue = watch("password", "");

const isEmailValid = useMemo(() => EMAIL_REGEX.test(emailValue), [emailValue]);
const isPasswordValidForProgress = useMemo(() => passwordValue &&
passwordValue.length >= 8, [passwordValue]);

const onSubmit = async (data) => {
  try {
    setLoading(true);
    const result = await loginUser({ email: data.email, password: data.password });

    if (!result || !result.token) {
      toast({
        title: "Login failed",
        description: "Invalid credentials",
        status: "error",
        duration: 3500,
        isClosable: true,
      });
      return;
    }

    // Store token
    localStorage.setItem("token", result.token);

    // Decode JWT
    const decodeJwt = (token) => {
      try {
        const base64Url = token.split(".")[1];
        const base64 = base64Url.replace(/-/g, "+").replace(/_/g, "/");
        const jsonPayload = decodeURIComponent(
          atob(base64)
            .split("")
            .map(c => "%" + ("00" + c.charCodeAt(0).toString(16)).slice(-2))
            .join("")
        );
        return JSON.parse(jsonPayload);
      }
    };
  }

```

```

    } catch {
      return {};
    }
  };

  const decoded = decodeJwt(result.token);
  const isOrgPresent = decoded?.is_org_present ?? false;
  const isAdmin = decoded?.is_admin ?? false;

  toast({
    title: "Logged in",
    description: "You have successfully logged in.",
    status: "success",
    duration: 2500,
    isClosable: true,
  });

  reset();

  // Navigate dynamically
  if (isOrgPresent) {
    navigate("/dashboard", { replace: true });
  } else if (isAdmin) {
    navigate("/orgRegister", { replace: true });
  } else {
    navigate("/dashboard", { replace: true });
  }

  if (onSubmitSuccess) onSubmitSuccess(result);
} catch (err) {
  console.error(err);
  toast({
    title: "Login failed",
    description: err.response?.data?.message || "Something went wrong",
    status: "error",
    duration: 3500,
    isClosable: true,
  });
} finally {
  setLoading(false);
}
};

return (
  <Flex flexDirection="column" justify="center" align="flex-start" w="100%"
fontFamily="Poppins">
    <Box bg="white" mt="25px" px={{ base: 6, md: 75 }} w="100%" borderRadius="8px"

```

```

boxSizing="border-box">
  <form onSubmit={handleSubmit(onSubmit)}>
    <Stack spacing={4}>
      <Box>
        <Heading fontSize="30px" mb={1} fontFamily="Poppins" fontWeight={700}
color="#000" lineHeight="40px">
          Welcome!
        </Heading>
        <Text color="#757575" fontSize="sm">Let's discover the HR solutions in a
Snap.</Text>
      </Box>

      {/* Email */}
      <FormControl id="email" isInvalid={!errors.email} mt={0}>
        <FormLabel fontSize="14px" color="#000">Email</FormLabel>
        <Input type="email" placeholder="Enter your email" {...register("email", {
          required: "Email is required",
          pattern: { value: EMAIL_REGEX, message: "Enter a valid email address" }
        })} {...inputStyle} />
        <FormErrorMessage>{errors.email?.message}</FormErrorMessage>
      </FormControl>

      {/* Password */}
      <FormControl id="password" isInvalid={!errors.password} mt={3}>
        <FormLabel fontSize="14px" color="#000">Password</FormLabel>
        <InputGroup>
          <Input type={showPassword ? "text" : "password"} placeholder="Enter your
Password" {...register("password", { required: "Password is required" })} {...inputStyle} />
          <InputRightElement h="full">
            <IconButton type="button" aria-label="Toggle password" icon={showPassword ?
<LuEyeOff /> : <LuEye />} onClick={() => setShowPassword(!showPassword)} variant="ghost"
size="sm" _hover={{ bg: "transparent" }} />
          </InputRightElement>
        </InputGroup>
        <FormErrorMessage>{errors.password?.message}</FormErrorMessage>
      </FormControl>

      {/* Remember + Forgot */}
      <Flex justify="space-between" align="center">
        <Checkbox {...register("remember")}>Remember me</Checkbox>
        <Link href="/forgot-password" color="#274167" fontSize="13px">Forgot
Password?</Link>
      </Flex>

      {/* Button */}
      <Box display="flex" justifyContent="flex-end">
        <Button type="submit" bg="#274167" color="white" fontFamily="Poppins"

```

```

isLoading={loading} isDisabled={!isValid || loading} mt={6}>
  Log In →
</Button>
</Box>
</Stack>
</form>
</Box>
</Flex>
);
}

```

✓ Notes:

1. This form is **fully Chakra UI styled**, like your server's.
 2. **No Redux needed**—calls your auth.js directly.
 3. Token is stored in **localStorage**, decoded for role checks.
 4. Navigation dynamically matches your server logic:
 - is_org_present → /dashboard
 - is_admin → /orgRegister
 - else → /dashboard
 5. Show/hide password and form validation preserved.
-

If you want, I can also **add the progress bar logic** like your server form and keep remember me fully functional, so it behaves **exactly like the server version** locally.

Do you want me to do that next?

i dnot need the chakra ui do without thta just give me the codes needed not onyl loginform

Ah, perfect! You want a **plain React version** of a local login that works exactly like your server form, **without Chakra UI**, and includes all the logic: calling the server API, storing the JWT, decoding it, and redirecting based on is_org_present and is_admin. I'll also include the minimal **axios setup** and auth.js functions.

Here's a complete, minimal setup:

1 api/axios.js – Axios instance

```
import axios from "axios";
```

```

const api = axios.create({
  baseURL: "http://66.116.197.167:5001/api/v1/hrm",
  headers: { "Content-Type": "application/json" },

```

```

    withCredentials: true, // send auth cookies if needed
  });

  // Automatically add token to all requests
  api.interceptors.request.use((config) => {
    const token = localStorage.getItem("token");
    if (token) config.headers.Authorization = `Bearer ${token}`;
    return config;
  });

export default api;

```

2 api/auth.js – Auth API calls

```

import api from "../axios";

export const loginUser = async (data) => {
  try {
    const response = await api.post("/auth/login_user", data);
    return response.data;
  } catch (err) {
    throw err.response?.data || err;
  }
};

```

3 components/LoginForm.js – Plain React login form

```

import { useState } from "react";
import { useNavigate } from "react-router-dom";
import { loginUser } from "../api/auth";

export default function LoginForm() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const navigate = useNavigate();

  const decodeJwt = (token) => {
    try {
      const base64Url = token.split(".")[1];
      const base64 = base64Url.replace(/-/g, "+").replace(/_/g, "/");
      const jsonPayload = decodeURIComponent(
        atob(base64)
          .split("")
          .map((c) => "%" + ("00" + c.charCodeAt(0).toString(16)).slice(-2))
      );
    }
  };

```



```

        .join("")
    );
    return JSON.parse(jsonPayload);
} catch {
    return {};
}
};

```

```

const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    setError("");

```

```

    try {
        const result = await loginUser({ email, password });

        if (!result.token) {
            setError("Invalid credentials");
            return;
        }

```

```

        // store token
        localStorage.setItem("token", result.token);

```

```

        const decoded = decodeJwt(result.token);
        const isOrgPresent = decoded?.is_org_present ?? false;
        const isAdmin = decoded?.is_admin ?? false;

```

```

        if (isOrgPresent) navigate("/dashboard");
        else if (isAdmin) navigate("/orgRegister");
        else navigate("/dashboard");
    } catch (err) {
        setError(err.message || "Login failed");
    } finally {
        setLoading(false);
    }
};

```

```

return (
    <div style={{ maxWidth: "400px", margin: "50px auto" }}>
        <h2>Login</h2>
        {error && <p style={{ color: "red" }}>{error}</p>}
        <form onSubmit={handleSubmit}>
            <div>
                <label>Email</label><br />
                <input
                    type="email"

```

```

      value={email}
      onChange={(e) => setEmail(e.target.value)}
      required
      style={{ width: "100%", padding: "8px", marginBottom: "10px" }}
    />
  </div>
  <div>
    <label>Password</label><br />
    <input
      type="password"
      value={password}
      onChange={(e) => setPassword(e.target.value)}
      required
      style={{ width: "100%", padding: "8px", marginBottom: "10px" }}
    />
  </div>
  <button type="submit" disabled={loading}>
    {loading ? "Logging in..." : "Login"}
  </button>
</form>
</div>
);
}

```

✓ How it works

1. User submits email/password.
2. loginUser calls your **server API**.
3. Token is stored in localStorage.
4. JWT is decoded to check is_org_present and is_admin.
5. Navigate:
 - Org exists → /dashboard
 - Admin → /orgRegister
 - Else → /dashboard

🔧 App.js – Setup routes

```

import { BrowserRouter as Router, Routes, Route } from "react-router-dom";
import LoginForm from "../components/LoginForm";

function Dashboard() {
  return <h2>Dashboard</h2>;
}

```

```

}

function OrgRegister() {
  return <h2>Organization Register</h2>;
}

export default function App() {
  return (
    <Router>
      <Routes>
        <Route path="/" element={<LoginForm />} />
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/orgRegister" element={<OrgRegister />} />
      </Routes>
    </Router>
  );
}

```

This gives you a **fully working local login** that communicates with your server. No Chakra UI, no Redux, minimal code, and fully functional navigation.

If you want, I can also add “**remember me**” **functionality** so the token persists and user auto-logs in when revisiting the page.

Do you want me to add that?

install react router dom

Absolutely! Since your app uses React Router v6 (based on the code I gave), you can install it via npm or yarn.

Using npm:

npm install react-router-dom

Using yarn:

yarn add react-router-dom

After installing, make sure your **main entry file** (like main.jsx or index.jsx) wraps your app with <BrowserRouter> if not already done:

```

import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { BrowserRouter } from "react-router-dom";

```

```
ReactDOM.createRoot(document.getElementById("root")).render(  
  <React.StrictMode>  
    <BrowserRouter>  
      <App />  
    </BrowserRouter>  
  </React.StrictMode>  
);
```

This ensures routing works properly for `/`, `/dashboard`, `/orgRegister`, etc.

If you want, I can now **add “remember me” and auto-login** so your local duplicate behaves exactly like the server. Do you want me to do that next?

Uncaught Error: `useNavigate()` may be used only in the context of a `<Router>` component. at invariant (react-router-dom.js?v=88b23cad:643:11) at `useNavigateUnstable` (react-router-dom.js?v=88b23cad:6236:3) at `useNavigate` (react-router-dom.js?v=88b23cad:6233:46) at `LoginForm` (LoginForm.jsx:10:20) at `Object.react_stack_bottom_frame` (react-dom_client.js?v=88b23cad:18509:20) at `renderWithHooks` (react-dom_client.js?v=88b23cad:5654:24) at `updateFunctionComponent` (react-dom_client.js?v=88b23cad:7475:21) at `beginWork` (react-dom_client.js?v=88b23cad:8525:20) at `runWithFiberInDEV` (react-dom_client.js?v=88b23cad:997:72) at `performUnitOfWork` (react-dom_client.js?v=88b23cad:12561:98) Understand this error react-dom_client.js?v=88b23cad:6966 An error occurred in the `<LoginForm>` component. Consider adding an error boundary to your tree to customize error handling behavior. Visit <https://react.dev/link/error-boundaries> to learn more about error boundaries. why this

Ah! This error is **very specific** and actually common when using React Router. The message says:

Uncaught Error: `useNavigate()` may be used only in the context of a `<Router>` component.

Here's what's happening:

- Your `LoginForm` calls `const navigate = useNavigate()`.
 - `useNavigate` **only works if the component is inside a Router**, like `<BrowserRouter>` or `<HashRouter>`.
 - If you try to render `LoginForm` in isolation (e.g., `<LoginForm />`) **without wrapping it inside a router**, React will throw this error.
-

How to fix it

1 Make sure your app root is wrapped in a `<BrowserRouter>`:

```
// main.jsx or index.jsx  
import React from "react";  
import ReactDOM from "react-dom/client";
```

```
import App from "./App";
import { BrowserRouter } from "react-router-dom";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

2 Make sure LoginForm is rendered as part of a route:

```
// App.jsx
import { Routes, Route } from "react-router-dom";
import LoginForm from "./LoginForm";
import Dashboard from "./Dashboard";

export default function App() {
  return (
    <Routes>
      <Route path="/" element={<LoginForm />} />
      <Route path="/dashboard" element={<Dashboard />} />
    </Routes>
  );
}
```

✓ After this, `useNavigate()` will work fine inside `LoginForm`.

If you want, I can provide a **full local setup** for your login form and dashboard **without Chakra UI**, fully functional with `useNavigate`, token storage, and redirect, so it behaves like your server version.

Do you want me to do that?

after doing everything what you said when logging in i am getting cors error

Ah, now we're hitting **CORS** — that's a browser security feature. Let me break this down carefully:

You said your backend is at:

`http://66.116.197.167:5001/api/v1/hrm`

And your React app is running locally, probably something like:

`http://localhost:5173`

Because the **origins are different**, the browser blocks requests unless the server allows it via CORS headers. That's why you get a CORS error.

Solutions

1 Enable CORS on the server

The correct way is to tell your server to accept requests from your local React app. For example, if your backend is Node.js + Express:

```
// server.js or app.js
import express from "express";
import cors from "cors";

const app = express();

app.use(cors({
  origin: "http://localhost:5173", // your React app
  credentials: true, // allow cookies to be sent
}));
```

```
app.use(express.json());
```

```
// your routes here
```

- `credentials: true` is required if you want cookies (like your `auth_token`) to be sent.
- `origin` should match exactly your local frontend.

2 Make sure Axios sends credentials

You already did:

```
const api = axios.create({
  baseURL: import.meta.env.VITE_BASE_URL,
  headers: { "Content-Type": "application/json" },
  withCredentials: true, // send cookies
});
```

- ✓ That's correct. This tells the browser to include cookies.

3 Check the server response

The server must send the correct headers for CORS **and cookies**:

```
Access-Control-Allow-Origin: http://localhost:5173
Access-Control-Allow-Credentials: true
```

If it sends * for Access-Control-Allow-Origin, the browser **will reject cookies**.

Quick workaround for local dev (not for production)

You can use a **proxy in Vite** to avoid CORS entirely:

```
// vite.config.js
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      "/api": {
        target: "http://66.116.197.167:5001/api/v1/hrm",
        changeOrigin: true,
        secure: false,
        rewrite: (path) => path.replace(/^\/api/, ""),
      },
    },
  },
});
```

Then in your axios:

```
const api = axios.create({
  baseURL: "/api", // proxied to backend
  headers: { "Content-Type": "application/json" },
  withCredentials: true,
});
```

Now, your frontend talks to /api locally, Vite proxies it to your server, and CORS is bypassed.

If you want, I can **rewrite your local login setup** so that you can log in **without touching the server CORS settings**, fully functional with token storage and navigation.

Do you want me to do that?

```
currently this is my axios code
import axios from "axios";
const api = axios.create({
  baseURL: "http://66.116.197.167:5001/api/v1/hrm",
  headers: { "Content-Type": "application/json" },
  withCredentials: true, // send auth cookies if needed
}); // Automatically add token to all requests
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
export default api;

how do i implement this for proxy
const api = axios.create({
  baseURL: "/api", // proxied to backend
  headers: { "Content-Type": "application/json" },
  withCredentials: true,
});
```

Perfect! You're basically just **switching your axios base URL** to go through the Vite dev proxy instead of hitting the backend directly. Here's a step-by-step on how to do it correctly:

1 Update your axios instance

```
import axios from "axios";
```

```
const api = axios.create({
  baseURL: "/api", // use Vite proxy
  headers: { "Content-Type": "application/json" },
  withCredentials: true, // include cookies
});

// Automatically attach token from localStorage
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

```
export default api;
```

✓ Notice the only change is `baseURL: "/api"`.

2 Setup Vite proxy

In your `vite.config.js`:

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      "/api": {
        target: "http://66.116.197.167:5001/api/v1/hrm",
        changeOrigin: true,
        secure: false,
        rewrite: (path) => path.replace(/^\/api/, ""), // remove /api prefix
      },
    },
  },
});
```

Explanation:

- Requests to `/api/login_user` → proxied to `http://66.116.197.167:5001/api/v1/hrm/login_user`
 - `changeOrigin: true` fixes host headers so the server accepts it
 - `rewrite` strips the `/api` prefix because your backend routes don't start with `/api`
-

3 Use in your auth calls

Your auth file doesn't need changes. Example:

```
import api from './axios';
```

```
export const loginUser = async (data) => {  
  const response = await api.post("/auth/login_user", data);  
  return response.data;  
};
```

Notice we didn't touch the endpoint paths—they stay the same. The proxy handles the full URL behind the scenes.

4 How to test

- Start Vite dev server (`npm run dev`)
 - Open the console in your browser
 - Log in using the local form
 - You should no longer see CORS errors
 - Tokens and cookies will work as expected
-

If you want, I can **give you a fully working local React login form** that logs in, stores the token, and redirects to the server dashboard **without using Chakra UI**, ready to drop into your project.

Do you want me to do that next?

Failed to load resource: the server responded with a status of 400 (Bad Request) why?

Ah! A **400 Bad Request** usually means your **request reached the server**, but the server didn't like what it got. Since you already have `withCredentials` and proxy setup, the problem is likely **data formatting or missing fields**.

Let's troubleshoot carefully.

1 Check what the server expects

Your backend login endpoint `/auth/login_user` probably expects a JSON payload like:

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

- Make sure your frontend is sending **exactly these keys**.
 - No extra nesting like `{ loginData: { email, password } }` unless the backend explicitly wants it.
-

2 Check axios request

If you're doing this:

```
const response = await api.post("/auth/login_user", loginData);
```

- `loginData` should be:

```
{
  email: "kumkum.dutta@delostylestudio.com",
  password: "yourpassword"
}
```

- Do **not** wrap it inside another object like `{ data: loginData }`. That will cause 400 Bad Request.
-

3 Check headers

You already have:

```
headers: { "Content-Type": "application/json" }
```

- ✓ That's correct. Make sure nothing overrides it elsewhere.
-

4 Check proxy rewrite

In `vite.config.js` you have:

```
rewrite: (path) => path.replace(/^\/api/, "")
```

- So `/api/auth/login_user` becomes `/auth/login_user` on the backend.
 - Double-check that matches your server route.
-

5 Debug the request

Add a console log before sending:

```
console.log("Login request payload:", loginData);
const response = await api.post("/auth/login_user", loginData);
```

- Copy the payload exactly and try **Postman** or **curl** with the same endpoint.
- If Postman works, the issue is your frontend formatting.

💡 Common mistakes that trigger **400**:

1. Sending undefined or empty string for email or password.
2. Wrapping payload in an extra object.
3. Misspelling field names (Email vs email).
4. Backend expecting application/x-www-form-urlencoded instead of JSON (less common in modern apps).

If you want, I can rewrite your **local login form + axios call** to **match exactly your server's payload**, so logging in locally works and goes straight to the server dashboard.

Do you want me to do that?

as you said in here "rewrite: (path) => path.replace(/^\/api/, '')" So /api/auth/login_user becomes /auth/login_user on the backend. Double-check that matches your server route." in myt server's auth file the code is import api from "/src/api/axios.js"; export const registerAdmin = async (data) => { try { const response = await api.post("/auth/register_admin", data); return response.data; } catch (error) { console.log(error); throw error; } }; export const registerOrg = async (data) => { try { const response = await api.post("/auth/register_org", data); return response.data; } catch (error) { console.log(error); throw error; } }; export const loginUser = async (data) => { const response = await api.post("/auth/login_user", data); return response.data; };

Perfect — this confirms that your backend **expects** the login request at /auth/login_user, exactly as you thought.

So with a Vite proxy like this:

```
// vite.config.js
export default defineConfig({
  server: {
    proxy: {
      '/api': {
        target: 'http://66.116.197.167:5001/api/v1/hrm',
        changeOrigin: true,
        rewrite: (path) => path.replace(/^\/api/, ''), // removes /api prefix
      },
    },
  },
});
```

- When your frontend calls:

```
api.post("/api/auth/login_user", data);
```

- Vite **rewrites it to**:

```
http://66.116.197.167:5001/api/v1/hrm/auth/login_user
```

✅ That matches your server route.

⚠️ Common 400 Causes With Proxy

1. Payload mismatch

Make sure data is **exactly**:

```
{  
  email: "kumkum.dutta@delostylestudio.com",  
  password: "yourpassword"  
}
```

- Not { loginData: { ... } }
- Not undefined fields

2. Headers / Content-Type

Your axios has:

```
headers: { "Content-Type": "application/json" }
```

- That's correct; backend expects JSON.

3. withCredentials mismatch

- If your backend sets cookies on a **different domain** than your frontend (localhost:5173 vs 66.116.197.167), withCredentials: true **requires CORS headers** on the server:

```
res.setHeader("Access-Control-Allow-Origin", "http://localhost:5173");  
res.setHeader("Access-Control-Allow-Credentials", "true");
```

Otherwise cookies won't be sent and login may fail.

✅ Recommended Frontend Call

```
import api from './axios';
```

```
const login = async (email, password) => {  
  try {  
    const response = await api.post('/api/auth/login_user', { email, password });  
    console.log('Server response:', response.data);  
    localStorage.setItem('token', response.data.token);  
  }  
}
```

```

    return response.data;
  } catch (err) {
    console.error('Login error', err.response?.data || err.message);
    throw err;
  }
};

```

- Note the **/api prefix**, because Vite proxy will rewrite it.
- Send **plain object** with email and password.

If you want, I can rewrite your **local login form + axios** to **fully match your server** and handle token storage & navigation **without Chakra UI**, so you can login locally and go to the server dashboard without errors.

Do you want me to do that next?

```

this is my server loginform data import { Box, Button, Checkbox, Flex, FormControl,
FormLabel, Heading, Input, InputGroup, InputRightElement, Link, Stack, Text, IconButton,
Image, FormErrorMessage, Progress, useToast, } from "@chakra-ui/react"; import { Navigate,
Link as RouterLink } from "react-router-dom"; import { useState, useMemo } from "react";
import { ViewIcon } from "@chakra-ui/icons"; import hide from "../assets/hide.png";
import { useForm } from "react-hook-form"; import { useDispatch, useSelector } from "react-
redux"; import { loginUserThunk } from "../slices/authSlice"; import { useNavigate } from
"react-router-dom"; import { LuEye } from "react-icons/lu"; import { LuEyeOff } from "react-
icons/lu"; import Cookies from "js-cookie"; const EMAIL_REGEX = /^[a-zA-Z0-9_.+-]+@[a-zA-
Z0-9-]+\.[a-zA-Z0-9-.]+\$/; export default function LoginForm({ onSubmitSuccess } = {}) { const
[showPassword, setShowPassword] = useState(false); const toast = useToast(); const
dispatch = useDispatch(); const navigate = useNavigate(); const { loading, error } =
useSelector((state) => state.auth); const token = Cookies.get("auth_token"); const inputStyle
= { fontFamily: "Poppins", fontSize: "15px", _placeholder: { color: "gray.400", fontFamily:
"Poppins", fontSize: "14px", }, h: "55px", borderWidth: "1.5px", borderColor: "#D0D5DD",
borderRadius: "8px", }; // if (token) { // return <Navigate to="/dashboard" replace />; // } //
react-hook-form config const { register, handleSubmit, watch, reset, formState: { errors,
isSubmitting, isValid }, } = useForm({ mode: "onChange", criteriaMode: "all", defaultValues: {
email: "", password: "", remember: false, }, }); // watch input values const emailValue =
watch("email", ""); const passwordValue = watch("password", ""); // helper checks for "field
is valid" used for progress calculation const isEmailValid = useMemo( () =>
EMAIL_REGEX.test(emailValue), [emailValue] ); const isPasswordValidForProgress =
useMemo( () => passwordValue && passwordValue.length >= 8, [passwordValue] ); //
compute progress percent (2 inputs: email + password) const progressPct = useMemo(() => {
const total = 2; const completed = (isEmailValid ? 1 : 0) + (isPasswordValidForProgress ? 1 :
0); return Math.round((completed / total) * 100); }, [isEmailValid,
isPasswordValidForProgress]); // API integration and show data in console const onSubmit =
async (data) => { try { const loginData = { email: data.email, password: data.password }; const
result = await dispatch(loginUserThunk(loginData)); if (result.meta.requestStatus ===
"fulfilled") { toast({ title: "Logged in", description: "You have successfully logged in.", status:
"success", duration: 2500, isClosable: true, }); reset(); const token = result.payload?.token ||

```

```

result.payload?.data?.token || result.payload; // decode JWT payload (safe base64url
decode) const decodeJwtPayload = (t) => { try { if (!t || typeof t !== "string") return null;
const base64Url = t.split(".")[1]; if (!base64Url) return null; const base64 =
base64Url.replace(/-/g, "+").replace(/_/g, "/"); const padded = base64 + "=".repeat((4 -
(base64.length % 4)) % 4); const jsonPayload = decodeURIComponent( atob(padded)
.split("").map( (c) => "%" + ("00" + c.charCodeAt(0).toString(16)).slice(-2) ).join("")); return
JSON.parse(jsonPayload); } catch (e) { return null; } }; // store token if present if (token &&
typeof token === "string") { try { localStorage.setItem("token", token); } catch (e) {
console.error("Failed to store token in localStorage:", e); } } const decoded =
decodeJwtPayload(token); const isOrgPresent = decoded?.is_org_present ??
decoded?.isOrgPresent ?? false; const isAdmin = decoded?.is_admin ?? decoded?.isAdmin
?? false; if (isOrgPresent) { navigate("/dashboard", { replace: true }); } else { // Only admins
should be allowed to create an organization profile. if (isAdmin) {
import(".././pages/Dashboard"); import(".././layouts/AppLayout");
import(".././pages/OrganizationRegister"); navigate("/orgRegister", { replace: true }); } else {
// Non-admin employees should be redirected to dashboard and informed
navigate("/dashboard", { replace: true }); } } } else { navigate("/orgRegister"); } if
(onSubmitSuccess) onSubmitSuccess(result.payload); // Optionally, redirect or store token
here } else { toast({ title: "Login failed", description: result.payload?.message || "Invalid
credentials.", status: "error", duration: 3500, isClosable: true, }); } } catch (err) { toast({ title:
>Login failed", description: "Something went wrong. Please try again.", status: "error",
duration: 3500, isClosable: true, }); } }; return ( <Flex flexDirection="column" justify="center"
align="flex-start" w="100%" fontFamily={"Poppins"} > { /* Previously there was a progress
bar here but we are currently not rendering it.. */ <Box bg="white" mt="25px" px={{ base:
6, md: 75 }} w="100%" borderRadius="8px" boxSizing="border-box" > { /* Use a real <form>
so Enter submits */ <form onSubmit={handleSubmit(onSubmit)}> <Stack spacing={4}>
<Box> <Heading // fontSize={{ "20px", "22px", "24px" }} fontSize="30px" mb={1}
fontFamily={"Poppins", sans-serif} fontWeight={700} color="#000000" lineHeight="40px" >
Welcome! </Heading> <Text color="#757575" fontSize="sm" fontFamily={"Poppins", sans-
serif" > Let's discover the HR solutions in a Snap. </Text> </Box> { /* Email */ <FormControl
id="email" isValid={!errors.email} mt={0}> <FormLabel fontFamily={"Poppins", sans-serif"
fontWeight="500" fontSize="14px" color="#000000" > Email </FormLabel> <Input
type="email" placeholder="Enter your email" borderRadius={4} fontSize="14px"
{...register("email", { required: "Email is required", pattern: { value: EMAIL_REGEX, message:
"Enter a valid email address", }, }}) aria-invalid={errors.email ? "true" : "false"} {...inputStyle}
/> <FormErrorMessage fontSize="12px" fontFamily={"poppins"} fontWeight={"500"}>
{errors.email?.message} </FormErrorMessage> </FormControl> { /* Password */
<FormControl id="password" isValid={!errors.password} mt={3}> <FormLabel
fontFamily={"Poppins", sans-serif} fontWeight="500" fontSize="14px" color="#000000" >
Password </FormLabel> <InputGroup> <Input type={showPassword ? "text" : "password"}
placeholder="Enter your Password" borderRadius={4} fontSize="14px"
{...register("password", { required: "Password is required", }}) aria-invalid={errors.password ?
"true" : "false"} {...inputStyle} /> <InputRightElement h="full"> <IconButton type="button" //
prevent this icon from submitting the form aria-label={ showPassword ? "Hide password" :
"Show password" } icon={showPassword ? <LuEyeOff /> : <LuEye />} onClick={() =>
setShowPassword((s) => !s)} variant="ghost" size="sm" _hover={{ bg: "transparent" }}
_active={{ bg: "transparent" }} /> </InputRightElement> </InputGroup> { /* Password hint /

```

```

errors */} {errors.password?.types ? ( <Box mt={2}>
{Object.values(errors.password.types).map((msg, idx) => ( <Text key={idx} color="red.500"
fontSize="12px" fontFamily={'poppins'} fontWeight={'500'}> {msg} </Text> ))) </Box> ) : (
<FormErrorMessage fontSize={'12px'} fontFamily={'poppins'}
fontWeight={'500'}>{errors.password?.message}</FormErrorMessage> )} </FormControl>
{/* Remember + Forgot */} <Flex justify="space-between" align="center"> <Checkbox
colorScheme="blue" {...register("remember")} sx={{ ".chakra-checkbox__control": { w:
"20px", h: "20px", }, }} > <Text fontSize="13px" fontFamily="Poppins", sans-serif"
fontWeight="500" color="#274167" > Remember me </Text> </Checkbox> <Link
as={RouterLink} to="/forgot-password" color="#274167" fontSize="13px"
fontFamily="Poppins", sans-serif" fontWeight="500" textDecoration="underline" > Forgot
Password? </Link> </Flex> { /* Button */} <Box display="flex" justifyContent="flex-end">
<Button type="submit" bg="#274167" color="white" fontFamily="Poppins", sans-serif"
fontWeight="500" fontSize="14px" _active={{ bg: "#1f3143" }} _hover={{ bg: "#1f3143" }}
rightIcon={<span>→</span>} isLoading={loading} isDisabled={!isValid || loading} aria-
disabled={!isValid || loading} mt={6} mb={-4} > Log In </Button> </Box> </Stack> </form>
</Box> </Flex> ); } and this my login form wait i am providing my local login code and then u
will bve able to knoe if there is a problem with payload

```

Perfect, I understand — let's go step by step. Go ahead and provide your **local login form code**, and then I can compare it with your server login form. That way, we can see exactly why the payload or request might be causing the 400 Bad Request error.

```

import { useState } from "react"; import { useNavigate } from "react-router-dom"; import {
loginUser } from "../api/auth"; export default function LoginForm() { const [email, setEmail] =
useState(""); const [password, setPassword] = useState(""); const [loading, setLoading] =
useState(false); const [error, setError] = useState(""); const navigate = useNavigate(); const
decodeJwt = (token) => { try { const base64Url = token.split(".")[1]; const base64 =
base64Url.replace(/-/g, "+").replace(/_/g, "/"); const jsonPayload = decodeURIComponent(
atob(base64).split("").map((c) => "%" + ("00" + c.charCodeAt(0).toString(16)).slice(-2))
.join("")); return JSON.parse(jsonPayload); } catch { return {}; } }; const handleSubmit = async
(e) => { e.preventDefault(); setLoading(true); setError(""); try { const result = await
loginUser({ email, password }); if (!result.token) { setError("Invalid credentials"); return; } //
store token localStorage.setItem("token", result.token); const decoded =
decodeJwt(result.token); const isOrgPresent = decoded?.is_org_present ?? false; const
isAdmin = decoded?.is_admin ?? false; if (isOrgPresent) navigate("/dashboard"); else if
(isAdmin) navigate("/orgRegister"); else navigate("/dashboard"); } catch (err) {
setError(err.message || "Login failed"); } finally { setLoading(false); } }; return ( <div style={{
maxWidth: "400px", margin: "50px auto" }}> <h2>Login</h2> {error && <p style={{ color:
"red" }}>{error}</p>} <form onSubmit={handleSubmit}> <div> <label>Email</label><br />
<input type="email" value={email} onChange={(e) => setEmail(e.target.value)} required
style={{ width: "100%", padding: "8px", marginBottom: "10px" }} /> </div> <div>
<label>Password</label><br /> <input type="password" value={password} onChange={(e) =>
setPassword(e.target.value)} required style={{ width: "100%", padding: "8px",
marginBottom: "10px" }} /> </div> <button type="submit" disabled={loading}> {loading ?
"Logging in..." : "Login"} </button> </form> </div> ); }

```

Perfect — now we can compare your **server login form vs local login form**. Here's what I notice that could explain your 400 Bad Request error:

1. Payload shape

Your server expects:

```
const response = await api.post("/auth/login_user", data);
```

And your server login form constructs:

```
const loginData = { email: data.email, password: data.password };
```

✅ That matches the backend.

Your local login form sends:

```
const result = await loginUser({ email, password });
```

Which should also match — so far so good.

2. Axios instance differences

Your **server login form** uses:

```
import api from "/src/api/axios.js";
```

And your axios.js has:

```
const api = axios.create({
  baseURL: "http://66.116.197.167:5001/api/v1/hrm",
  headers: { "Content-Type": "application/json" },
  withCredentials: true,
});
```

✅ This works for cross-domain cookies if your backend has Access-Control-Allow-Credentials: true and the cookie domain matches.

Your **local login** uses loginUser:

```
export const loginUser = async (data) => {
  const response = await api.post("/auth/login_user", data);
  return response.data;
};
```

✅ That's correct.

3. CORS + Cookies

A common cause of 400 (or CORS issues) is:

- The frontend is running on `http://localhost:5173`
- The backend is `http://66.116.197.167:5001`

- withCredentials: true is set, but the backend must also allow:

Access-Control-Allow-Origin: http://localhost:5173

Access-Control-Allow-Credentials: true

If the backend doesn't send these headers, the browser blocks the request or the cookie isn't sent properly. Sometimes the backend interprets the missing token/cookie as invalid and returns 400.

4. proxy vs direct request

If you try:

```
const api = axios.create({
  baseURL: "/api",
  headers: { "Content-Type": "application/json" },
  withCredentials: true,
});
```

You need a **dev proxy** in vite.config.js:

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': {
        target: 'http://66.116.197.167:5001/api/v1/hrm',
        changeOrigin: true,
        secure: false,
        rewrite: (path) => path.replace(/^\/api/, ''),
      },
    },
  },
});
```

Then /api/auth/login_user will be sent as /auth/login_user to your backend.

✅ Double-check your backend route is /api/v1/hrm/auth/login_user or just /auth/login_user.

5. Debugging tip

Open **DevTools** → **Network tab**:

- Check the **request payload**: make sure { email, password } is being sent.
- Check the **response body**: your backend may be sending more detail than just 400.

- Check **request headers**: is the Content-Type: application/json correct? Is the token missing?
-

◆ Likely cause of 400

- **Backend doesn't recognize the request payload** or headers (e.g., missing Content-Type or malformed JSON).
 - **CORS with credentials** misconfigured — the browser may strip cookies or headers, backend returns 400.
-

If you want, I can rewrite your **local login form + axios instance + Vite proxy** in a fully working version that matches your server so you can log in **locally** without CORS or 400 issues.

Do you want me to do that next?

how do i know which of the git is connected

Ah! You want to check **which remote Git repository your local project is connected to**. Here's how you do it step by step:

1. Check Git remotes

Open your terminal in the project folder and run:

```
git remote -v
```

Example output:

```
origin git@github.com:username/project.git (fetch)
```

```
origin git@github.com:username/project.git (push)
```

- origin → default remote name (usually the one you cloned from)
 - The URL (git@github.com:username/project.git or https://github.com/...) shows which repository it's connected to.
 - (fetch) is for pulling, (push) is for pushing changes.
-

2. Check the current branch

```
git branch
```

- The * indicates the current branch.
 - Often remotes have a branch with the same name (origin/main, origin/master, etc.).
-

3. Check branch tracking

git status

- It will show something like:

On branch main

Your branch is up to date with 'origin/main'.

- This tells you which remote branch your local branch is linked to.
-

4. Optional: Get full remote info

git remote show origin

Example output:

```
* remote origin
Fetch URL: git@github.com:username/project.git
Push URL: git@github.com:username/project.git
HEAD branch: main
Remote branches:
  main tracked
  dev tracked
Local branch configured for 'git pull':
  main merges with remote main
Local ref configured for 'git push':
  main pushes to main (up to date)
```

This gives a detailed view of all tracked branches and URLs.

If you want, I can give you a **quick one-liner** that tells you **exactly which GitHub repo your current folder is connected to** without all the extra info.

Do you want me to do that?

We use cookies

export const { addItem, increaseQty, decreaseQty, removeItem, } = cartSlice.actions; export default cartSlice.reducer; why at last of each slide these statements are given and does each slice have three keys namely name, initialState and reducer and all have object value and the entire create slice is an object?

In a typical Redux Toolkit slice:

```
const cartSlice = createSlice({
  name: "cart",
  initialState: {
    items: [],
```

```
},  
reducers: {  
  addItem: (state, action) => {},  
  increaseQty: (state, action) => {},  
  decreaseQty: (state, action) => {},  
  removeItem: (state, action) => {},  
},  
});
```

createSlice() returns an **object**.

That returned object usually contains:

- name
- reducer
- actions
- caseReducers
- etc.

So yes — the whole cartSlice is an object.

Why these lines are written at the end

```
export const {  
  addItem,  
  increaseQty,  
  decreaseQty,  
  removeItem,  
} = cartSlice.actions;
```

```
export default cartSlice.reducer;
```

1. cartSlice.actions

Redux Toolkit automatically creates **action creators** from your reducers.

This:

```
reducers: {  
  addItem: (state, action) => {}  
}
```

automatically generates:

```
cartSlice.actions.addItem
```

which is a function.

So this line:

```
export const { addItem } = cartSlice.actions;
```

is just **object destructuring**.

Equivalent to:

```
export const addItem = cartSlice.actions.addItem;
```

You export them so other components can dispatch actions:

```
dispatch(addItem(product));
```

2. export default cartSlice.reducer

cartSlice.reducer is the actual reducer function generated by Redux Toolkit.

You export it because it must be added to the Redux store:

```
import cartReducer from "./cartSlice";
```

```
const store = configureStore({  
  reducer: {  
    cart: cartReducer,  
  },  
});
```

Does every slice have only 3 keys?

Not exactly.

In createSlice, the commonly used configuration keys are:

```
createSlice({  
  name,  
  initialState,  
  reducers,  
})
```

These are the main required ones.

But there can also be others like:

extraReducers

Example:

```
createSlice({  
  name: "cart",  
  initialState,  
  reducers: {},  
  extraReducers: (builder) => {}  
})
```

Important distinction

Input to createSlice

This object:

```
{
  name,
  initialState,
  reducers
}
```

is the configuration object you pass INTO createSlice.

Output from createSlice

cartSlice itself becomes another object:

```
{
  name,
  reducer,
  actions,
  caseReducers,
  ...
}
```

So there are two different objects involved.

Visual flow

```
createSlice(configurationObject)
  ↓
returns
  ↓
sliceObject
```

Example:

```
const cartSlice = createSlice({
  name: "cart",
  initialState: {},
  reducers: {}
});
```

- The parameter is a config object
- The return value is a slice object

They are not the same object internally.

reducers: { addItem: (state, action) => {} } in here action has type nad payload as default in redux?

Yes — in Redux Toolkit, the action object in a reducer always follows a standard structure.

When you write:

```
reducers: {  
  addItem: (state, action) => {}  
}
```

the action automatically has (at minimum) these fields:

```
{  
  type: "sliceName/reducerName",  
  payload: ...  
}
```

1. What is action in Redux Toolkit?

Every dispatched action is a plain object:

```
{  
  type: "cart/addItem",  
  payload: someValue  
}
```

So yes — **type and payload are standard conventions**, but technically:

- type → always required (Redux core requirement)
 - payload → convention used by Redux Toolkit
-

2. Where do type and payload come from?

✓ type

Automatically generated from:

slice name + reducer name

Example:

```
createSlice({  
  name: "cart",  
  reducers: {  
    addItem: ...  
  }  
})
```

Generates:

type: "cart/addItem"

✓ payload

Automatically filled when you dispatch an action creator:

```
dispatch(addItem(product));
```

Redux Toolkit converts it into:

```
{
  type: "cart/addItem",
  payload: product
}
```

3. Full structure of action

In Redux Toolkit, an action looks like:

```
{
  type: string,
  payload?: any,
  meta?: any,
  error?: boolean
}
```

But in most everyday usage, you only deal with:

- type
- payload

4. Important clarification

👉 Redux itself does NOT force payload

Redux core only requires:

```
{ type: "something" }
```

But Redux Toolkit standardizes actions so that:

- function arguments → become payload
- errors → go into error
- extra info → goes into meta

5. Example in a slice

```
addItem: (state, action) => {
  console.log(action.type); // "cart/addItem"
```



```
console.log(action.payload); // whatever you passed
}
```

6. Summary

- ✓ action.type → always exists (required by Redux)
 - ✓ action.payload → default convention in Redux Toolkit
 - ✓ Redux Toolkit automatically creates both when using createSlice
 - ✓ You don't manually define them
-

If you want, I can also show how payload is created internally using createAction so you understand what Redux Toolkit is doing under the hood.

```
import { createSlice } from "@reduxjs/toolkit"; const cartSlice = createSlice({ name: "cart",
initialState: { items: [], }, reducers: { addItem: (state, action) => { const existing =
state.items.find(i => i.id === action.payload.id); if (existing) { existing.qty += 1; } else {
state.items.push({ ...action.payload, qty: 1 }); }, removeItem: (state, action) => { state.items
= state.items.filter(i => i.id !== action.payload); }, increaseQty: (state, action) => { const item
= state.items.find(i => i.id === action.payload); if (item) item.qty += 1; }, decreaseQty: (state,
action) => { const item = state.items.find(i => i.id === action.payload); if (item && item.qty >
1) item.qty -= 1; }, }, }); export const { addItem, removeItem, increaseQty, decreaseQty, } =
cartSlice.actions; export default cartSlice.reducer; how state have access to id and whats
different between find and filter why used find first and then filter?
```

state itself does **not** have an id.

Your state looks like this:

```
{
  items: []
}
```

Each object inside items has an id, for example:

```
items: [
  {
    id: 1,
    title: "Phone",
    price: 200,
    qty: 1
  }
]
```

So when you write:

```
state.items.find(i => i.id === action.payload.id)
```

i represents each item object inside items, and i.id accesses that item's id.

Why find() is used in addItem

```
const existing = state.items.find(i => i.id === action.payload.id);
```

find() searches for **one matching item**.

Example:

```
const arr = [  
  { id: 1 },  
  { id: 2 }  
];
```

```
arr.find(i => i.id === 2);
```

Result:

```
{ id: 2 }
```

If not found:

```
undefined
```

Here it's used to check:

"Does this product already exist in cart?"

If yes:

```
existing.qty += 1;
```

Otherwise:

```
state.items.push({ ...action.payload, qty: 1 });
```

Why filter() is used in removeItem

```
state.items = state.items.filter(i => i.id !== action.payload);
```

filter() returns a **new array** containing all items that pass the condition.

Example:

```
const arr = [  
  { id: 1 },  
  { id: 2 },  
  { id: 3 }  
];
```

```
arr.filter(i => i.id !== 2);
```

Result:

```
[
  { id: 1 },
  { id: 3 }
]
```

So here it means:

"Keep every item except the one whose id matches."

That's why filter() is perfect for deleting items.

Main Difference

Method	Returns	Used For
find()	Single item	Searching/checking existence
filter()	New array	Removing/selecting multiple items

In Your Slice

find() used when:

You need a specific item to update quantity.

```
const item = state.items.find(...)
```

filter() used when:

You want a new array without some item.

```
state.items = state.items.filter(...)
```

Simple Real-Life Analogy

Imagine a bag of products.

find()

You look for:

"Where is the product with id 5?"

You get ONE product.

filter()

You say:

"Give me all products except id 5."

You get a NEW collection without that product.

so when we make a search bar in react in which page with api student data are called do we use find to search for a particular student or employee with role

Yes — for a search bar in React, whether you use find() or filter() depends on what you want.

Use find() when:

You want **only one result**.

Example:

- Search by unique id
- Find one employee

```
const student = students.find(s => s.id === 101);
```

Returns:

```
{ id: 101, name: "Rahul" }
```

Use filter() when:

You want **multiple matching results**.

This is what search bars usually use.

Example:

- Search by name
- Search by role
- Search employees containing text

```
const results = students.filter(s =>
  s.name.toLowerCase().includes(searchText.toLowerCase())
);
```

If search text is "ra":

Result:

```
[
  { name: "Rahul" },
  { name: "Rakesh" }
]
```

Searching employees by role

Example data:

```
const employees = [
  { id: 1, name: "Aman", role: "developer" },
  { id: 2, name: "Sara", role: "designer" },
  { id: 3, name: "John", role: "developer" }
];
```

Using filter():

```
const developers = employees.filter(
  emp => emp.role === "developer"
);
```

Result:

```
[
  { id: 1, name: "Aman", role: "developer" },
  { id: 3, name: "John", role: "developer" }
]
```

In React Search Bar

Usually:

```
const filteredStudents = students.filter(student =>
  student.name.toLowerCase().includes(search.toLowerCase())
);
```

Then render:

```
{filteredStudents.map(student => (
  <p key={student.id}>{student.name}</p>
))}
```

Quick Rule

Situation	Method
Need one exact item	find()
Need list of matching items	filter()
Search bar	Usually filter()

Most search bars use filter() because many items can match the search text.

so why in quantity at first find is used why not in both case filter?

Because in quantity operations (increaseQty, decreaseQty) you only need **one specific item object** to modify.

That's why `find()` is better.

Example

Suppose cart has:

```
items = [  
  { id: 1, name: "Phone", qty: 1 },  
  { id: 2, name: "Laptop", qty: 2 }  
]
```

When user clicks:

`increaseQty(2)`

you only want:

```
{ id: 2, name: "Laptop", qty: 2 }
```

So:

```
const item = state.items.find(i => i.id === action.payload);
```

Then:

```
item.qty += 1;
```

Simple and efficient.

Why NOT `filter()` here?

Because `filter()` always returns an ARRAY.

Example:

```
const item = state.items.filter(i => i.id === 2);
```

Result:

```
[  
  { id: 2, name: "Laptop", qty: 2 }  
]
```

Now you don't have the object directly.

You'd need:

```
item[0].qty += 1;
```

That is unnecessary because:

- only one item should match
- `find()` directly gives the object

Main Logic

Quantity Update

You want:

"Give me the exact item so I can edit it."

Use:

`find()`

Remove Item

You want:

"Create a new array without this item."

Use:

`filter()`

Think Like This

Operation	Needed Result	Best Method
Update one item	Single object	<code>find()</code>
Delete item(s)	New filtered array	<code>filter()</code>

Technically Could You Use `filter()`?

Yes:

```
const item = state.items.filter(i => i.id === action.payload)[0];
```

But it's bad practice because:

- `filter()` checks ALL items
- returns an array unnecessarily
- less readable
- slower for this use case

`find()` stops as soon as it finds the match.

okay make a cartslice with a ui of 4 items card each card will have add to cart button which will add them and then it will have a navbar and footer the navbar will have cart logo that redirect to cart page where each item can be added in quantity and decrease too and also

removed completely first make it using find and then using filter i want to know the difference and u will tell me where i can see the difference

Folder Structure

```
src/
├── app/
│   └── store.js
├── features/
│   └── cart/
│       ├── cartSliceFind.js
│       └── cartSliceFilter.js
├── components/
│   ├── Navbar.jsx
│   ├── Footer.jsx
│   └── ProductCard.jsx
├── pages/
│   ├── Home.jsx
│   └── Cart.jsx
├── App.jsx
└── main.jsx
```

1. Install Packages

```
npm install react-redux @reduxjs/toolkit react-router-dom
```

2. store.js

```
import { configureStore } from "@reduxjs/toolkit";
import cartReducer from "../features/cart/cartSliceFind";
```

```
export const store = configureStore({
  reducer: {
    cart: cartReducer,
  },
});
```

Later you can replace:

cartSliceFind

with:

cartSliceFilter

to compare both versions.

3. cartSliceFind.js (Using FIND)

```
import { createSlice } from "@reduxjs/toolkit";
```

```
const cartSlice = createSlice({  
  name: "cart",
```

```
  initialState: {  
    items: [],  
  },
```

```
  reducers: {  
    addItem: (state, action) => {  
      const existing = state.items.find(  
        item => item.id === action.payload.id  
      );
```

```
      if (existing) {  
        existing.qty += 1;  
      } else {  
        state.items.push({  
          ...action.payload,  
          qty: 1,  
        });  
      }  
    },
```

```
    increaseQty: (state, action) => {  
      const item = state.items.find(  
        item => item.id === action.payload  
      );
```

```
      if (item) {  
        item.qty += 1;  
      }  
    },
```

```
    decreaseQty: (state, action) => {  
      const item = state.items.find(  
        item => item.id === action.payload  
      );
```

```
      if (item && item.qty > 1) {  
        item.qty -= 1;  
      }  
    }  
  }  
});
```

```

    },

    removeItem: (state, action) => {
      state.items = state.items.filter(
        item => item.id !== action.payload
      );
    },
  },
});

export const {
  addItem,
  increaseQty,
  decreaseQty,
  removeItem,
} = cartSlice.actions;

export default cartSlice.reducer;

```

4. cartSliceFilter.js (Using FILTER)

```

import { createSlice } from "@reduxjs/toolkit";

const cartSlice = createSlice({
  name: "cart",

  initialState: {
    items: [],
  },

  reducers: {
    addItem: (state, action) => {

      const existing = state.items.filter(
        item => item.id === action.payload.id
      )[0];

      if (existing) {
        existing.qty += 1;
      } else {
        state.items.push({
          ...action.payload,
          qty: 1,
        });
      }
    },
  },
});

```

```

increaseQty: (state, action) => {

  const item = state.items.filter(
    item => item.id === action.payload
  )[0];

  if (item) {
    item.qty += 1;
  }
},

decreaseQty: (state, action) => {

  const item = state.items.filter(
    item => item.id === action.payload
  )[0];

  if (item && item.qty > 1) {
    item.qty -= 1;
  }
},

removeItem: (state, action) => {
  state.items = state.items.filter(
    item => item.id !== action.payload
  );
},
},
});

export const {
  addItem,
  increaseQty,
  decreaseQty,
  removeItem,
} = cartSlice.actions;

export default cartSlice.reducer;

```

5. Navbar.jsx

```

import { Link } from "react-router-dom";
import { useSelector } from "react-redux";

const Navbar = () => {

  const cartItems = useSelector(state => state.cart.items);

```

```

return (
  <nav
    style={{
      padding: "20px",
      background: "black",
      color: "white",
      display: "flex",
      justifyContent: "space-between",
    }}
  >
    <Link to="/" style={{ color: "white" }}>
      Home
    </Link>

    <Link to="/cart" style={{ color: "white" }}>
      Cart ({cartItems.length})
    </Link>
  </nav>
);
};

export default Navbar;

```

6. Footer.jsx

```

const Footer = () => {
  return (
    <footer
      style={{
        padding: "20px",
        background: "black",
        color: "white",
        marginTop: "40px",
        textAlign: "center",
      }}
    >
      Footer
    </footer>
  );
};

export default Footer;

```

7. ProductCard.jsx

```

import { useDispatch } from "react-redux";
import { addItem } from "../features/cart/cartSliceFind";

const ProductCard = ({ product }) => {

  const dispatch = useDispatch();

  return (
    <div
      style={{
        border: "1px solid gray",
        padding: "20px",
        width: "200px",
      }}
    >
      <h3>{product.name}</h3>

      <p>${product.price}</p>

      <button
        onClick={() => dispatch(addItem(product))}
      >
        Add To Cart
      </button>
    </div>
  );
};

export default ProductCard;

```

8. Home.jsx

```

import ProductCard from "../components/ProductCard";

const products = [
  { id: 1, name: "Phone", price: 500 },
  { id: 2, name: "Laptop", price: 1000 },
  { id: 3, name: "Watch", price: 200 },
  { id: 4, name: "Headphones", price: 150 },
];

const Home = () => {
  return (
    <div
      style={{
        display: "flex",
        gap: "20px",

```

```

        padding: "20px",
      }}
    >
    {products.map(product => (
      <ProductCard
        key={product.id}
        product={product}
      />
    ))}
  </div>
);
};

```

```
export default Home;
```

9. Cart.jsx

```
import { useDispatch, useSelector } from "react-redux";
```

```
import {
  increaseQty,
  decreaseQty,
  removeItem,
} from "../features/cart/cartSliceFind";
```

```
const Cart = () => {
```

```
  const dispatch = useDispatch();
```

```
  const items = useSelector(
    state => state.cart.items
  );
```

```
  return (
    <div style={{ padding: "20px" }}>
```

```
      <h2>Cart Page</h2>
```

```
      {items.map(item => (
```

```
        <div
          key={item.id}
          style={{
            border: "1px solid gray",
            marginBottom: "20px",
            padding: "20px",
          }}

```

```

    >
    <h3>{item.name}</h3>

    <p>Price: ${item.price}</p>

    <p>Qty: {item.qty}</p>

    <button
      onClick={() =>
        dispatch(increaseQty(item.id))
      }
    >
      +
    </button>

    <button
      onClick={() =>
        dispatch(decreaseQty(item.id))
      }
    >
      -
    </button>

    <button
      onClick={() =>
        dispatch(removeItem(item.id))
      }
    >
      Remove
    </button>

  </div>
  )}}
</div>
);
};

export default Cart;

```

10. App.jsx

```

import { BrowserRouter, Routes, Route } from "react-router-dom";

import Navbar from "../components/Navbar";
import Footer from "../components/Footer";

import Home from "../pages/Home";

```

```
import Cart from "../pages/Cart";

const App = () => {
  return (
    <BrowserRouter>

      <Navbar />

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/cart" element={<Cart />} />
      </Routes>

      <Footer />

    </BrowserRouter>
  );
};

export default App;
```

11. main.jsx

```
import ReactDOM from "react-dom/client";
import App from "../App";

import { Provider } from "react-redux";
import { store } from "../app/store";

ReactDOM.createRoot(
  document.getElementById("root")
).render(

  <Provider store={store}>
    <App />
  </Provider>
);
```

WHERE YOU WILL SEE THE DIFFERENCE

You asked the most important part.

Version 1 → FIND

```
const item = state.items.find(...)
```


What happens internally?

- stops searching immediately after finding match
- returns ONE object
- faster
- cleaner

Open console and add:

```
console.log("find running");
```

inside reducers.

You'll notice fewer iterations in large arrays.

Version 2 → FILTER

```
const item = state.items.filter(...)[0]
```

What happens internally?

- checks EVERY item
- creates NEW ARRAY
- then takes first item
- unnecessary work

Even if item found early,
filter() still continues.

BEST WAY TO VISUALLY SEE DIFFERENCE

Add logs:

FIND VERSION

```
const item = state.items.find(i => {  
  console.log("find checking", i.id);  
  return i.id === action.payload;  
});
```

FILTER VERSION

```
const item = state.items.filter(i => {  
  console.log("filter checking", i.id);  
  return i.id === action.payload;  
})[0];
```

Now:

- add many products
- increase quantity

You'll see:

FIND

Stops early.

Example:

find checking 1

find checking 2

Stops.

FILTER

filter checking 1

filter checking 2

filter checking 3

filter checking 4

Checks ALL items every time.

FINAL UNDERSTANDING

find()

Use when:

- updating one item
- searching unique item
- need object directly

filter()

Use when:

- removing items
- getting multiple matches
- creating new filtered array

That's why professional Redux cart logic almost always uses:

- find() for quantity updates

- `filter()` for remove operations

`item => item.id === action.payload )[0];` why this syntax why 0 end

Because `filter()` always returns an **array**.

So this:

```
state.items.filter(  
  item => item.id === action.payload  
)
```

returns something like:

```
[  
  { id: 2, name: "Laptop", qty: 1 }  
]
```

Even if only one item matches, it is still inside an array.

Why [0]?

[0] means:

"Give me the first element of the array"

So:

```
const item = state.items.filter(  
  item => item.id === action.payload  
)[0];
```

becomes:

```
const item = {  
  id: 2,  
  name: "Laptop",  
  qty: 1  
};
```

Now you can do:

```
item.qty += 1;
```

Without [0]

You would get:

```
[  
  { id: 2, name: "Laptop", qty: 1 }  
]
```

Then this would fail:

```
item.qty += 1;
```

because arrays do not have qty.

You would need:

```
item[0].qty += 1;
```

Compare Clearly

find()

```
const item = state.items.find(  
  item => item.id === action.payload  
);
```

Returns directly:

```
{ id: 2, qty: 1 }
```

filter()

```
const item = state.items.filter(  
  item => item.id === action.payload  
);
```

Returns:

```
[  
  { id: 2, qty: 1 }  
]
```

So to access object:

```
item[0]
```

Real Meaning of [0]

Array indexes start from 0.

Example:

```
const arr = ["a", "b", "c"];
```

Index Value

0 "a"

1 "b"

2 "c"

So:

`arr[0]`

returns:

`"a"`

Why This Shows `find()` Is Better

Because:

`find()`

already returns the object directly.

No need for:

`[0]`

No unnecessary array creation.

Cleaner and faster.

if two of same id then what would do find?

Good question — this is where the *real difference between `find()` and `filter()` becomes important*.

If there are TWO items with the same id

Example cart state:

```
items = [  
  { id: 1, name: "Phone", qty: 1 },  
  { id: 1, name: "Phone", qty: 2 }  
]
```

What `find()` does

```
const item = state.items.find(i => i.id === 1);
```

Result:

```
{ id: 1, name: "Phone", qty: 1 }
```

👉 It returns **ONLY the first match**

👉 It ignores the rest

What `filter()` does

```
const items = state.items.filter(i => i.id === 1);
```

Result:

```
[
  { id: 1, name: "Phone", qty: 1 },
  { id: 1, name: "Phone", qty: 2 }
]
```

👉 It returns **ALL matches**

So what happens in real cart apps?

In a **proper cart design**, this situation should NEVER happen.

Because:

Each product should have a unique id

You should not store duplicate products as separate rows

Instead, you store like this:

```
[
  { id: 1, name: "Phone", qty: 3 }
]
```

So quantity increases instead of duplicating items.

Why find() is still correct for cart

Because in a good cart system:

- id is unique
- only ONE item per product exists
- quantity changes inside that item

So:

```
const item = state.items.find(i => i.id === action.payload);
```

makes perfect sense.

If duplicates exist (bad design case)

Then:

- find() → only updates first one ❌ (bug risk)
- filter() → gets all duplicates ✓

But instead of fixing logic with filter(), you should fix the **data structure**

Key takeaway

Case	Best method
IDs are unique (correct cart design)	find()
Multiple matches needed	filter()
Duplicate IDs exist	Data structure problem, not method problem

Simple way to remember

- find() → “give me ONE item”
 - filter() → “give me ALL matching items”
-

If you want, I can next show you:

👉 how big ecommerce apps *prevent duplicate cart items at architecture level* (very important interview topic)

export const { addItem, increaseQty, decreaseQty, removeItem, } = cartSlice.actions; why .actions suffix?

Because createSlice() from Redux Toolkit automatically generates an **actions object** for you.

What createSlice() actually returns

When you write:

```
const cartSlice = createSlice({
  name: "cart",
  initialState: {},
  reducers: {
    addItem: () => {},
    removeItem: () => {},
  }
});
```

Redux Toolkit internally creates something like:

```
cartSlice = {
  name: "cart",
  reducer: ..., // the main reducer function
  actions: {    // 👉 THIS is what you're using
    addItem: function,
    removeItem: function
  }
}
```

So why .actions?

Because:

All reducer functions you wrote are converted into **action creators** and stored inside actions.

So:

`cartSlice.actions`

means:

"Give me all the action creator functions generated from reducers"

Example

Your reducers:

```
reducers: {  
  addItem: (state, action) => {},  
  removeItem: (state, action) => {}  
}
```

Become:

```
cartSlice.actions = {  
  addItem: (payload) => ({ type: "cart/addItem", payload }),  
  removeItem: (payload) => ({ type: "cart/removeItem", payload })  
}
```

Redux Toolkit generates them automatically.

Why we destructure it

Instead of writing:

```
export const addItem = cartSlice.actions.addItem;  
export const removeItem = cartSlice.actions.removeItem;
```

we write:

```
export const {  
  addItem,  
  removeItem  
} = cartSlice.actions;
```

👉 cleaner

👉 shorter

👉 same result

Simple mental model

Think of:

`cartSlice.actions`

as a **box of buttons**

Each button = one action creator

So:

- `addItem` button
 - `removeItem` button
 - `increaseQty` button
-

Important clarity

There are 2 different things:

Part	Meaning
reducers	logic inside slice
actions	functions you dispatch in UI

Flow in real app

1. You click button:

```
dispatch(addItem(product))
```

2. Redux Toolkit creates action:

```
{ type: "cart/addItem", payload: product }
```

3. Reducer runs:

```
addItem(state, action)
```

One-line answer

We use `.actions` because Redux Toolkit automatically converts your reducers into **action creator functions and stores them inside the actions object**.

If you want, I can next show you:

👉 how Redux Toolkit automatically creates action types like `"cart/addItem"` (this is usually asked in interviews)

cartSlice = { name: "cart", reducer: ..., // the main reducer function actions: { // 🙌 THIS is what you're using addItem: function, removeItem: function } } so redux internally creates this? so reducer which we write as a key and its value as object is converted into string and there comes another key automatically called actions with objewcrt value which originally has actions

You're very close in understanding the flow, but one part needs correction.

Redux Toolkit does **not convert your reducer object into strings**, and it does **not "replace" it with an actions key**. Instead, it builds a structured object from what you define.

Let's break it cleanly.

What YOU write

```
const cartSlice = createSlice({
  name: "cart",
  initialState: {},
  reducers: {
    addItem: (state, action) => {},
    removeItem: (state, action) => {}
  }
});
```

What Redux Toolkit creates internally

It creates a slice object like this:

```
cartSlice = {
  name: "cart",

  reducer: function rootReducer(state, action) {
    // handles all reducer cases internally
  },

  actions: {
    addItem: (payload) => ({
      type: "cart/addItem",
      payload
    }),

    removeItem: (payload) => ({
      type: "cart/removeItem",
      payload
    })
  }
}
```

Key corrections to your assumption

✗ Not correct:

reducers object is converted into strings

✓ Correct:

Each reducer function is used to generate:

1. A reducer case inside ONE combined reducer function
2. An action creator function inside actions

What actually happens internally

Step 1: Redux Toolkit reads your reducers

```
reducers: {  
  addItem: fn,  
  removeItem: fn  
}
```

Step 2: It creates action types

Automatically generates strings like:

```
"cart/addItem"  
"cart/removeItem"
```

Format:

```
sliceName/reducerName
```

Step 3: It creates action creators

```
addItem: (payload) => ({  
  type: "cart/addItem",  
  payload  
})
```

Step 4: It creates ONE reducer function

Instead of separate reducers, it builds a single internal reducer:

```
function reducer(state, action) {  
  switch(action.type) {  
    case "cart/addItem":
```

```
// run your logic
break;

case "cart/removeItem":
  // run your logic
  break;
}
}
```

So the real structure is:

```
cartSlice = {
  name: "cart",
  reducer: function,
  actions: object
}
```

Nothing is “converted into strings inside your reducer object”. Strings are only used as **action types** internally.

Simple mental model

Think of createSlice as a factory:

You give it:

```
reducers: {
  addItem: logic,
  removeItem: logic
}
```

It gives you:

1. A big reducer machine

```
cartSlice.reducer
```

2. Action buttons

```
cartSlice.actions.addItem()
cartSlice.actions.removeItem()
```

3. Auto-generated action types

```
"cart/addItem"
"cart/removeItem"
```

Final clean takeaway

- Your reducers object is NOT converted to strings

- Each reducer becomes:
 - a case inside a combined reducer function
 - an action creator inside actions
 - Redux Toolkit generates action type strings internally like "sliceName/reducerName"
-

If you want, I can next show you:

👉 how createSlice replaces traditional switch(action.type) reducer code (this makes Redux Toolkit much easier to understand).

cartSlice = { name: "cart", reducer: function rootReducer(state, action) { // handles all reducer cases internally }, actions: { addItem: (payload) => ({ type: "cart/addItem", payload }), removeItem: (payload) => ({ type: "cart/removeItem", payload }) } } then why outside why actions not inside the function as stated

Good catch — this confusion is very common, and the key idea is:

👉 **reducer and actions are two completely different outputs of createSlice(). They are not nested inside each other.**

Why actions is NOT inside the reducer function

Because they do totally different jobs:

1. reducer (the function)

This is used by Redux internally.

It:

- receives (state, action)
- updates state based on action.type
- runs inside the Redux store

Think:

“This is the brain that updates state”

2. actions (object of functions)

These are used by YOU in React components.

They:

- create action objects
- are passed into dispatch()

Think:

“These are buttons you click in UI”

They belong to different places in the system

Flow architecture:

```
React UI
  ↓ dispatch(addItem())
actions object (action creator)
  ↓
{ type: "cart/addItem", payload }
  ↓
reducer function
  ↓
state updated
  ↓
UI re-renders
```

Why they are NOT inside the reducer function

Because:

✗ Reducer does NOT create actions

It only reacts to actions

✗ Actions do NOT update state directly

They only describe what happened

If actions were inside reducer (why that would be wrong)

If we imagined:

```
reducer: function() {
  actions: { ... }
}
```

That would mean:

- UI depends on reducer internals ✗
 - dispatch logic mixed with state logic ✗
 - Redux design breaks separation of concerns ✗
-

Redux design principle (very important)

Redux enforces:

Part Responsibility

Actions Describe what happened

Reducer Decide how state changes

Store Holds state

UI Dispatches actions

Each is independent.

Why createSlice returns them separately

Because it builds 3 independent outputs:

```
{  
  name,  
  reducer, // for store  
  actions // for UI  
}
```

So you can use them separately:

Store:

```
configureStore({  
  reducer: cartSlice.reducer  
})
```

UI:

```
dispatch(addItem(product))
```

Simple analogy

Think of a restaurant:

- **Actions = menu items (order button)**
- **Reducer = kitchen (prepares food)**
- **Store = inventory system**

Menu is NOT inside kitchen.

Kitchen does NOT create menu.

They are connected, but separate.

Final clarity

- 👉 actions is outside because it is meant for the UI layer
 - 👉 reducer is separate because it is meant for the state layer
 - 👉 Redux Toolkit connects them internally, but keeps them independent
-

If you want next, I can show you:

👉 the *exact internal transformation of createSlice into plain Redux (without Toolkit)* — this makes everything click instantly.

```
import { createSlice } from "@reduxjs/toolkit"; const cartSlice = createSlice({ name: "cart",
initialState: { items: [], }, reducers: { addItem: (state, action) => { const existing =
state.items.filter( item => item.id === action.payload.id )[0]; if (existing) { existing.qty += 1; }
else { state.items.push({ ...action.payload, qty: 1, }); }, increaseQty: (state, action) => { //
const item = state.items.filter( // item => item.id === action.payload // )[0]; const item =
state.items.filter(i => { console.log("filter checking", i.id); return i.id === action.payload; })[0];
if (item) { item.qty += 1; }, decreaseQty: (state, action) => { const item = state.items.filter(
item => item.id === action.payload )[0]; if (item && item.qty > 1) { item.qty -= 1; },
removeItem: (state, action) => { state.items = state.items.filter( item => item.id !==
action.payload ); }, }, }); export const { addItem, increaseQty, decreaseQty, removeItem, } =
cartSlice.actions; export default cartSlice.reducer; in the code which one is reducer which
are actions?
```

In your code, everything inside reducers: { ... } are **reducers**, and everything you export from cartSlice.actions are **actions (action creators)**.

Let's break it clearly.

1. These are reducers (logic functions)

Inside createSlice:

```
reducers: {
  addItem: (state, action) => {},
  increaseQty: (state, action) => {},
  decreaseQty: (state, action) => {},
  removeItem: (state, action) => {}
}
```

👉 Each of these is a **reducer function**

They:

- take (state, action)
- modify state
- run inside Redux when an action is dispatched

So these are reducers:

- ✓ addItem
 - ✓ increaseQty
 - ✓ decreaseQty
 - ✓ removeItem
-

2. These are actions (action creators)

At the bottom:

```
export const {
  addItem,
  increaseQty,
  decreaseQty,
  removeItem,
} = cartSlice.actions;
```

👉 These are NOT reducers anymore

👉 These are **functions that create actions**

Example:

addItem(product)

returns:

```
{
  type: "cart/addItem",
  payload: product
}
```

3. Simple mapping

Name in code	Type	Role
addItem (inside reducers)	reducer	updates state
addItem (exported)	action creator	sends action
increaseQty (inside reducers)	reducer	updates state
increaseQty (exported)	action creator	sends action

Same name, but TWO different roles.

4. Full flow (important)

Example:

dispatch(addItem(product))

Step-by-step:

Step 1: action creator runs

`addItem(product)`

produces:

```
{  
  type: "cart/addItem",  
  payload: product  
}
```

Step 2: reducer runs

Redux finds:

`addItem: (state, action) => { ... }`

and executes it.

Step 3: state updates

Your cart changes.

5. Very important confusion fix

Even though names are same:

```
reducers: {  
  addItem: ...  
}
```

and

```
export const { addItem } = cartSlice.actions;
```

They are NOT the same thing.

6. Mental model

Think like this:

Inside slice = "logic rules"

`addItem` → HOW state changes

Outside export = "buttons"

`addItem()` → SEND request to change state

Final answer

Reducers in your code:

- addItem
- increaseQty
- decreaseQty
- removeItem

Actions in your code:

- addItem
- increaseQty
- decreaseQty
- removeItem (from cartSlice.actions)

If you want, I can next draw a **real memory diagram of Redux flow (UI → action → reducer → store)** so you can visualize it permanently.

why actions are exported?

Actions are exported because they are the **only things your React components need in order to change Redux state**.

Reducers stay inside Redux. Actions go outside to the UI.

Simple idea first

👉 **Reducer = logic inside Redux**

👉 **Actions = functions you use in React (UI)**

So you export actions so React can use them.

What happens if you DON'T export actions?

Then in your component you cannot do:

```
dispatch(addItem(product))
```

Because addItem would be inaccessible outside the slice file.

So nothing from UI can trigger state updates.

Why actions must be exported

Because React components live in a different layer:

1. Redux slice file

Defines logic:

```
addItem: (state, action) => { ... }
```

This is internal logic.

2. React component

Needs a way to say:

“Hey Redux, add this item”

So you use:

```
dispatch(addItem(product))
```

But for that, you must export addItem.

What cartSlice.actions really is

Redux Toolkit automatically creates this:

```
cartSlice.actions = {  
  addItem: function,  
  increaseQty: function,  
  decreaseQty: function,  
  removeItem: function  
}
```

Each one creates an action object like:

```
{  
  type: "cart/addItem",  
  payload: product  
}
```

Why export individually instead of whole object?

Instead of:

```
export default cartSlice.actions;
```

we do:

```
export const { addItem } = cartSlice.actions;
```

Because it allows:

- cleaner imports
 - tree-shaking (unused code removed)
 - better readability
-

Flow of why export is needed

React button click



dispatch(addItem())



action object created



Redux reducer runs



state updates

Without exporting actions:

- ✗ React cannot dispatch anything
 - ✗ Redux becomes unusable from UI
-

Real-life analogy

Think of Redux like a kitchen:

- **Reducer = chef (does cooking)**
- **Actions = order tickets (request from customer)**

You export actions because:

Customers (React UI) need order tickets to tell chef what to cook.

Without exporting actions:

- 👉 customers cannot place orders
-

Final answer

- 👉 Actions are exported because they are the **bridge between React UI and Redux state**
 - 👉 They let components dispatch changes to the store
 - 👉 Without exporting them, Redux would be isolated and unusable from UI
-

If you want, I can next show you:

- 👉 what actually happens when you call dispatch(addItem()) step-by-step in memory (this clears Redux completely).

so exported actions what do they do send the actions to slice where reducer tells how to change state and store the payload?

You're very close — just a small correction in wording and structure.

Let's make it precise.

What exported actions actually do

When you export and use an action like:

```
dispatch(addItem(product))
```

👉 the exported action **does NOT directly send anything to the slice**

Instead, it **creates an action object**.

Step-by-step flow (important)

1. You call the exported action

```
addItem(product)
```

This is an **action creator function**.

It returns:

```
{
  type: "cart/addItem",
  payload: product
}
```

2. You dispatch it

```
dispatch(addItem(product))
```

Now Redux receives:

```
{ type: "cart/addItem", payload: product }
```

3. Redux Store receives the action

The **store** is the central manager.

It:

- sees the type
 - finds matching reducer case
-

4. Slice reducer runs

Inside your slice:

```
addItem: (state, action) => {  
  // logic here  
}
```

Redux calls it like:

```
addItem(currentState, actionObject)
```

5. State is updated

Inside reducer:

```
state.items.push(...)
```

Redux Toolkit uses Immer internally, so mutations are safe.

Correct version of your understanding (fixed)

You said:

exported actions send actions to slice where reducer tells how to change state and store payload

✓ Almost correct, but better phrased:

- 👉 Exported actions **create an action object**
 - 👉 dispatch sends that action to the Redux store
 - 👉 Store routes it to the correct reducer
 - 👉 Reducer updates state using action.payload
-

Clean mental model

UI button click



exported action (addItem)



creates action object



dispatch sends to store



store matches type



slice reducer runs



state updated

Key correction

- ✗ Actions do NOT directly talk to reducers
- ✓ Store is the middleman

Simple analogy

Think of it like a food delivery system:

- **Action creator (addItem)** → creates order slip
- **dispatch** → delivery boy
- **store** → restaurant reception
- **reducer** → chef
- **state** → prepared food inventory

Final answer

- 👉 Exported actions only **create action objects**
- 👉 dispatch sends them to the Redux store
- 👉 The store forwards them to the correct reducer
- 👉 Reducer uses action.payload to update state

If you want, I can next show you:

- 👉 a real memory-level diagram of Redux flow (this makes everything click instantly).